AI Engineering Handbook

Part 8 — LLM Engineering, Embeddings & RAG

Contents

1 Chapter 8.1 — What Is an LLM? 6
1.1 From Traditional ML to LLMs . . . 6
1.2 Core Vocabulary . . . 6

2 Chapter 8.2 — LLM Architecture Overview 8
2.1 Stage by Stage . . . 8

3 Chapter 8.3 — Open-Source LLM Ecosystem 10
3.1 Open Source vs. Open Weights vs. Source-Available . . . 10
3.2 Major Model Families (practical orientation, not a training-data dump) . . . 10
3.3 What to Evaluate Per Family . . . 11

4 Chapter 8.4 — Model Selection 12
4.1 Selection Factors . . . 12
4.2 Decision Matrix (template — fill in per project) . . . 12

5 Chapter 8.5 — Local LLM Inference 14
5.1 Practical Toolchain . . . 14
5.2 Key Concepts . . . 14
5.2.1 Quick Pipeline Example . . . 14
5.2.2 LAB 1 — Load a Local Open-Weight LLM . . . 15

6 Chapter 8.6 — Hugging Face Model Loading 16
6.1 `from_pretrained` Parameters Worth Knowing . . . 16
6.2 Gated Models and Authentication . . . 16

7 Chapter 8.7 — Text Generation 18
7.1 Parameters . . . 18
7.1.1 LAB 2 — Compare Decoding Strategies . . . 19

8 Chapter 8.8 — Decoding Strategies 20

9 Chapter 8.9 — Chat Models & Chat Templates 22
9.1 Roles . . . 22
9.2 Why Manual Concatenation Fails . . . 22

10 Chapter 8.10 — Prompt Engineering 24
10.1 Core Techniques . . . 24
10.2 Before / After . . . 24

11 Chapter 8.11 — Prompt Design Patterns 26
11.0.1 Classification Prompt Example . . . 26
11.0.2 Extraction Prompt Example . . . 27

11.0.3 Grounded Prompt Example . . . 27

12 Chapter 8.12 — Structured Outputs 28
12.1 Approaches . . . 28
12.1.1 Pydantic Preview . . . 28

13 Chapter 8.13 — LLM API Engineering 30
13.1 Core Patterns . . . 30

14 Chapter 8.14 — Streaming LLM Responses 32
14.1 Concepts . . . 32
14.1.1 Minimal FastAPI Streaming Endpoint . . . 32

15 Chapter 8.15 — Embeddings 34
15.1 Core Concepts . . . 34
15.2 Embedding Model ≠ Generative LLM . . . 34
15.3 Practical Example — Sentence Transformers . . . 35
15.4 Practical Example — Raw Hugging Face . . . 35

16 Chapter 8.16 — Embedding Model Selection 36
16.1 What to Weigh . . . 36

17 Chapter 8.17 — Cosine Similarity for Semantic Search 38
17.1 From Scratch (NumPy) . . . 38
17.2 With scikit-learn . . . 38
17.3 With Sentence Transformers . . . 39

18 Chapter 8.18 — Vector Database Fundamentals 40
18.1 Core Concepts . . . 40
18.2 Comparison . . . 40

19 Chapter 8.19 — FAISS 42
19.1 Core Index Types . . . 42
19.2 Persistence . . . 43
19.3 L2 Distance vs. Inner Product vs. Cosine Similarity . . . 43
19.3.1 LAB 4 — Implement FAISS Search . . . 43

20 Chapter 8.20 — Vector Database Project 44
20.1 Pipeline . . . 44
20.2 Environment Specification . . . 45
20.3 Evaluation . . . 45

21 Chapter 8.21 — Document Ingestion 46
21.1 Extraction vs. Retrieval . . . 46
21.2 Format-by-Format Notes . . . 46

22 Chapter 8.22 — Document Chunking 48
22.1 Strategies . . . 48
22.1.1 Recursive Splitting Example . . . 48

23 Chapter 8.23 — Chunking Experiment Lab 50
23.0.1 LAB 5 — Experiment With Chunk Sizes . . . 50

24 Chapter 8.24 — Metadata 51
24.1 Why Metadata Matters . . . 51

25 Chapter 8.25 — Retrieval **53**
25.1 Retrieval Strategies . . . 53
25.2 BM25 vs. Embeddings . . . 53
25.3 Hybrid Search Pattern . . . 54

26 Chapter 8.26 — Reranking **55**
26.1 Concepts . . . 55
26.2 Why Retrieve-More-Then-Rerank-Fewer Improves Quality . . . 56
26.2.1 LAB 8 — Add Reranking . . . 56

27 Chapter 8.27 — RAG Fundamentals **57**
27.1 Stage by Stage . . . 57

28 Chapter 8.28 — Basic RAG Implementation **59**
28.0.1 LAB 6 — Build RAG Without LangChain . . . 60

29 Chapter 8.29 — RAG with Citations **61**
29.1 Citation Mapping . . . 62

30 Chapter 8.30 — Advanced RAG Patterns **63**

31 Chapter 8.31 — RAG Failure Modes **64**
31.1 Common Failure Modes . . . 64
31.2 Diagnostic Decision Tree . . . 64

32 Chapter 8.32 — RAG Evaluation **66**
32.1 Retrieval Metrics . . . 66
32.2 Generation Metrics . . . 66
32.3 Building an Evaluation Dataset . . . 66

33 Chapter 8.33 — RAG Evaluation Project **68**

34 Chapter 8.34 — Hallucination Control **69**
34.1 Strategies . . . 69

35 Chapter 8.35 — Context Window Management **71**
35.1 Concepts . . . 71

36 Chapter 8.36 — LLM Cost & Latency Optimization **73**
36.1 Levers . . . 73
36.2 Measurement Framework . . . 73

37 Chapter 8.37 — LLM Security Basics **75**
37.1 Core Threats . . . 75
37.2 RAG-Specific Injection Path . . . 75
37.3 Defensive Patterns . . . 76

38 Chapter 8.38 — RAG with Authorization **77**
38.1 Why Filtering Only After Retrieval Is Dangerous . . . 77
38.2 Practical Prototype . . . 77

39 Chapter 8.39 — LLM Observability Preview **79**
39.1 Signals to Capture . . . 79

40 Chapter 8.40 — RAG Deployment **81**
40.1 Production Concerns . . . 82

40.1.1 LAB 10 — Deploy RAG with FastAPI 82

41 Chapter 8.41 — Dockerizing RAG **83**
41.1 What to Cover 83
41.1.1 LAB 11 — Dockerize the System 84

42 Chapter 8.42 — LLM Project 1: Semantic Search **85**

43 Chapter 8.43 — LLM Project 2: Document Q&A **86**

44 Chapter 8.44 — LLM Project 3: Multi-Document RAG **87**

45 Chapter 8.45 — LLM Project 4: RAG API **88**

46 Chapter 8.46 — LLM Project 5: Advanced RAG **89**

47 Chapter 8.47 — LLM Project Dataset & Environment Specifications **90**
47.1 Dataset Specification Template 90
47.2 Environment Specification Template 90
47.3 Reproducibility Deliverables 90

48 Chapter 8.48 — LLM/RAG Debugging **92**

49 Chapter 8.49 — LLM/RAG Performance Benchmarking **94**
49.1 What to Measure 94
49.2 Benchmark Script 94

50 Model Selection Decision Tree **96**
50.1 Prompting vs. RAG vs. Fine-Tuning vs. Agents 96
50.2 RAG vs. Long Context 97

51 Chapter 8.50 — LLM Engineering Interview Preparation **99**
51.1 LLMs & Tokenization 99
51.2 Inference & Generation Parameters 100
51.3 Chat Templates & Prompting 100
51.4 Structured Outputs 101
51.5 API Engineering 101
51.6 Streaming 101
51.7 Embeddings 101
51.8 Vector Search & FAISS 102
51.9 Vector Databases 102
51.10 Chunking 103
51.11 Metadata & Retrieval 103
51.12 Reranking 103
51.13 RAG Fundamentals 104
51.14 Citations 104
51.15 Advanced RAG Patterns 104
51.16 RAG Failure Modes & Debugging 105
51.17 Evaluation 105
51.18 Hallucination 105
51.19 Context Window Management 106
51.20 Security 106
51.21 RAG with Authorization 106
51.22 Observability 106
51.23 Deployment & Cost/Latency 107
51.24 FastAPI / Docker Deployment 107

51.25Debugging Scenarios 107
51.26System Design / Scenario-Based 108
51.27Coding 108
51.28More Conceptual / Rapid-Fire 109

52 Final Project Roadmap **116**

53 Transition to Part 9 **118**

54 Part 9 — LLM Fine-Tuning & PEFT **119**

55 Part 10 — Open-Source Model Fine-Tuning & Contribution Engineering **120**

56 Part 11 — Agentic AI, LangChain, LangGraph & MCP **121**

Chapter 1

Chapter 8.1 — What Is an LLM?

A **Large Language Model (LLM)** is a neural network trained on massive text corpora to predict the next token in a sequence. Everything an LLM appears to "do" — answering questions, writing code, summarizing — is an emergent consequence of this one training objective applied at scale.

1.1 From Traditional ML to LLMs

Approach	Input representation	Feature engineering	Generalization
Traditional ML (e.g. logistic regression, XGBoost)	Hand-built tabular features	Manual, task-specific	Narrow, per-task
Classical NLP (TF-IDF, bag-of-words, HMMs)	Sparse word statistics	Manual, linguistic	Narrow, per-task
Transformer models (BERT-era)	Learned dense embeddings	Learned via pretraining	Broad within NLP
LLMs (GPT-era, decoder-only, instruction-tuned)	Learned embeddings + massive pretraining	Learned, task-agnostic	Broad across tasks and domains

Part 7 covered *why* the Transformer architecture works. This chapter is about the practical vocabulary you need before you write code.

1.2 Core Vocabulary

- **Pretrained model** — trained on raw text with a self-supervised objective (next-token prediction). It completes text; it does not "follow instructions" reliably.
- **Base model** — a synonym for a pretrained model with no instruction-tuning or chat fine-tuning applied. Prompted with `"The capital of France is"`, a base model continues the sentence; it does not treat the prompt as a question posed by a user.
- **Instruction-tuned model** — a base model further trained on (instruction, response) pairs so it treats the prompt as a task to perform rather than text to continue.

- **Chat model** — an instruction-tuned model further trained (often with RLHF/DPO) on multi-turn conversations with explicit roles (`system`, `user`, `assistant`). Nearly every production LLM you call through an API today is a chat model.
- **Decoder-only model** — generates tokens autoregressively, left to right, attending only to previous tokens (causal attention). This is the architecture behind GPT-style models, Llama, Mistral, Qwen, Gemma. It is the dominant architecture for modern LLMs.
- **Encoder-decoder model** — an encoder reads the full input bidirectionally, a decoder generates output conditioned on the encoder's representation (e.g. T5, BART, and translation models). Less common for open-ended chat, still relevant for translation and some structured generation tasks.
- **Context window** — the maximum number of tokens (input + output combined, for most APIs) the model can attend to in a single call. A model with an 8K context window that receives a 7,500-token prompt has roughly 500 tokens left for its answer.
- **Tokens** — the atomic units a model actually operates on. Not words: a tokenizer typically splits `"unbelievable"` into subword pieces such as `un`, `believ`, `able`. Token count, not word count, is what you budget against context limits and API cost.
- **Inference** — the act of running a trained model forward to produce output, as opposed to *training*, which updates the model's weights.

• Common Mistake

Confusing "the weights are downloadable" with "this is a chat model I can prompt with a system message." Many downloadable checkpoints are base models. Feeding a base model a chat-style prompt with no chat template often produces rambling completions instead of answers – not because the model is bad, but because it was never trained to treat your text as an instruction.

Chapter 2

Chapter 8.2 — LLM Architecture Overview

You don't need to re-derive attention to build with LLMs, but you do need the practical shape of the pipeline, because every failure mode you'll debug in this Part maps onto one of these stages.

```
Text
 ↓
Tokenizer
 ↓
Token IDs
 ↓
Embeddings
 ↓
Transformer Blocks
 ↓
Logits
 ↓
Sampling / Decoding
 ↓
Generated Tokens
 ↓
Text
```

2.1 Stage by Stage

- **Tokenizer** — deterministic, non-neural code that maps raw text to a sequence of integer token IDs from a fixed vocabulary (commonly 32K–150K entries depending on the model family). Tokenizers are model-specific; using the wrong tokenizer for a model produces IDs the model was never trained on.
- **Token IDs** — integers, one per token, that index into the model's embedding table.
- **Embeddings** — each token ID is looked up in a learned embedding matrix, producing a dense vector (commonly 768–8192 dimensions depending on model size). This is the model's internal, geometric representation of "meaning" before any context is applied.
- **Transformer blocks** — stacked layers of self-attention and feed-forward networks that let each token's representation absorb information from other tokens in the sequence. This is where "context" is actually built.
- **Hidden states** — the vector representation of each token after passing through the transformer blocks. The final layer's hidden state for the last token is what decides what comes next.

- **Logits** — a raw, unnormalized score for every token in the vocabulary, produced by projecting the final hidden state through the model's output (language modeling) head.
- **Probability distribution** — logits passed through a softmax, turning raw scores into a probability over the entire vocabulary for "what token comes next."
- **Sampling / decoding** — the algorithm that picks the next token from that probability distribution (see Chapter 8.7–8.8). This step is *not* part of the neural network — it's ordinary code you control directly.
- **Generated tokens → text** — the chosen token ID is appended to the sequence, and the whole forward pass repeats for the next position (or the model stops). Once generation ends, the tokenizer decodes the ID sequence back into a string.

• Best Practice

When debugging "the model produced garbage," localize the stage first: is it a tokenizer mismatch (wrong tokenizer for the checkpoint), a decoding problem (temperature too high), or a prompting problem (no chat template applied)? Chapter 8.48 gives you a full symptom-to-cause table – but the mental model above is what lets you use it.

Chapter 3

Chapter 8.3 — Open-Source LLM Ecosystem

3.1 Open Source vs. Open Weights vs. Source-Available

These terms get used interchangeably in marketing copy, and that sloppiness causes real licensing mistakes.

- **Open weights** — the trained parameter files are freely downloadable. This says nothing about the training data, training code, or what you're legally allowed to do with the model. Most "open" LLMs today are open-weight, not open-source.
- **Open source (in the traditional software sense)** — the full pipeline is public and freely licensed: architecture code, training code, *and* training data (or a reproducible description of it), typically under an OSI-approved license. Very few LLMs meet this bar; most "open-source LLM" claims in industry writing are really about weights.
- **Source-available** — code or weights are visible for inspection but under a license with restrictions (commercial-use caps, field-of-use restrictions, no redistribution) that does not meet OSI's definition of open source.

• Common Mistake

Writing "Llama is open source" in a design doc and having a legal/compliance reviewer push back. Llama's weights are downloadable and its license is permissive for most uses, but the license carries specific conditions (e.g., historically a monthly-active-user threshold that triggers a separate commercial license). "Open weight, custom permissive license" is the accurate description – check the current license text for the exact model version you intend to ship, since terms vary by release.

3.2 Major Model Families (practical orientation, not a training-data dump)

Family	Typical sizes	Notable trait	Licensing posture
Llama (Meta)	~1B–405B	Broad ecosystem support, strong baseline	Custom permissive license with conditions
Mistral / Mixtral (Mistral AI)	~7B–8x22B (MoE)	Efficient dense and mixture-of-experts variants	Mix of Apache 2.0 and commercial-only releases — check per model

Family	Typical sizes	Notable trait	Licensing posture
Qwen (Alibaba)	~0.5B–110B+	Strong multilingual (esp. Chinese/English), competitive coding variants	Mostly Apache 2.0 / custom permissive, varies by release
Gemma (Google)	~2B–27B	Distilled from larger Gemini research, strong for its size class	Custom Gemma license (open-weight, some restrictions)
Phi (Microsoft)	~3.8B–14B	"Small language model" line, trained on heavily curated/synthetic data for strong reasoning-per-parameter	MIT / permissive on recent releases

For every family: **verify size, license, and context length against the specific checkpoint you plan to deploy** — these details change between point releases within the same family, and this table is a starting orientation, not a substitute for reading the model card.

3.3 What to Evaluate Per Family

- **Model size** — drives VRAM/RAM requirements and latency, not necessarily quality on your task.
- **Context length** — varies by checkpoint even within one family (a base 7B and a "long-context" variant of the same 7B can differ by 8x).
- **Strengths / weaknesses** — task-dependent (coding, multilingual, reasoning, instruction-following); always validate against your own eval set (Chapter 8.32), not marketing benchmarks.
- **Hardware considerations** — a 7B model in FP16 needs roughly 14GB VRAM just for weights, before activations and KV cache; quantization changes this substantially (Part 9 covers quantization mechanics in depth).
- **Licensing considerations** — commercial-use terms, redistribution terms, MAU/revenue thresholds if any.
- **Local inference suitability** — some checkpoints ship instruction-tuned and quantized variants specifically optimized for local/edge use; others target datacenter GPUs only.
- **API availability** — many open-weight models are *also* available hosted (via the model creator, or third-party inference providers), which changes the local-vs-API tradeoff in Chapter 8.4.

Chapter 4

Chapter 8.4 — Model Selection

There is no single "best" LLM. Model selection is a constrained optimization problem, and treating it as one is what separates engineering from vibes.

4.1 Selection Factors

- **Task** — classification, extraction, summarization, open-ended generation, code, agentic tool use — each has different model strengths.
- **Model quality** — measured against *your* evaluation set (Chapter 8.32), not a generic leaderboard.
- **Context length** — must exceed your prompt + retrieved context + expected output, with margin.
- **Latency** — larger models are slower per token; some deployments need first-token latency more than throughput, others the reverse.
- **RAM / VRAM** — hard constraint for local deployment; determines which model sizes and quantization levels are even loadable.
- **Model size** — correlates with capability and cost/latency, not linearly.
- **License** — commercial permissibility, redistribution rights.
- **Inference cost** — per-token API pricing, or amortized hardware cost for self-hosting.
- **Privacy** — data residency and confidentiality requirements may rule out API models entirely.
- **Deployment target** — edge device, on-prem server, cloud GPU, serverless API.
- **Multilingual support** — varies significantly between families and even between sizes within a family.
- **Tool-calling support** — not all instruction-tuned models are trained for structured function-calling; check the model card.
- **Structured-output support** — native JSON-mode / grammar-constrained decoding support varies by provider and by local-serving stack.

4.2 Decision Matrix (template — fill in per project)

Factor	Weight	Model A	Model B	Model C
Task quality (your eval)				
Context length				
Latency (p50/p95)				
Hardware fit				
License fit				

Factor	Weight	Model A	Model B	Model C
Cost per 1K requests				
Privacy fit				
Weighted total				

- **Best Practice**

Build the decision matrix with real numbers from your own eval set and your own latency measurements (Chapter 8.49), not from a vendor's published benchmark. Published benchmarks measure the vendor's task distribution, not yours.

Chapter 5

Chapter 8.5 — Local LLM Inference

Local inference means running a model's forward pass on hardware you control, without a network call to a hosted API.

5.1 Practical Toolchain

- **Hugging Face `transformers`** — the standard Python library for loading and running open-weight models; supports both CPU and GPU inference.
- **`pipeline`** — a high-level convenience wrapper that bundles tokenizer + model + generation config behind one call. Best for prototyping.
- **Direct model loading** (`AutoTokenizer` + `AutoModelForCausalLM`) — lower-level, gives you full control over generation parameters, batching, and device placement. This is what you want for production code.

5.2 Key Concepts

- **`device_map`** — controls which device (CPU, a specific GPU, or `"auto"` for automatic multi-GPU sharding) each part of the model is placed on.
- **`torch_dtype`** — the numeric precision used for weights and activations (`float32`, `float16`, `bfloat16`). Lower precision roughly halves memory use and often increases throughput, with a small, usually acceptable, quality cost.
- **CPU inference** — works for any model but is slow for anything beyond small models; viable for low-throughput or offline batch jobs.
- **GPU inference** — the default for anything beyond experimentation; VRAM is the binding constraint.
- **Batching** — processing multiple prompts in one forward pass to improve GPU utilization; requires padding and attention masks.
- **Generation configuration** — the set of decoding parameters (Chapter 8.7) that controls *how* tokens are chosen, independent of the model weights themselves.

5.2.1 Quick Pipeline Example

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
```

```
    model="Qwen/Qwen2.5-1.5B-Instruct",
    torch_dtype="auto",
    device_map="auto",
)

output = generator(
    "Explain the difference between a base model and a chat model.",
    max_new_tokens=200,
)
print(output[0]["generated_text"])
```

- **Common Mistake**

Loading a full-precision (`float32`) 7B model on a machine with 8GB of VRAM and getting a CUDA out-of-memory error. Roughly: `float32` needs about 4 bytes/parameter, `float16/bfloat16` about 2 bytes/parameter, before accounting for activations and KV cache. A 7B model in `float32` alone needs about 28GB just for weights.

5.2.2 LAB 1 — Load a Local Open-Weight LLM

Load a small instruction-tuned model (1–3B parameters) locally using `transformers`. Generate a response to three different prompts. Record: load time, VRAM/RAM used, tokens/sec.

Chapter 6

Chapter 8.6 — Hugging Face Model Loading

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_id = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
```

6.1 `from_pretrained` Parameters Worth Knowing

- **`torch_dtype`** — numeric precision at load time; `"auto"` reads the checkpoint's recommended dtype from its config.
- **`device_map`** — `"auto"` lets `accelerate` decide device placement, including sharding a model across multiple GPUs or spilling to CPU/disk when VRAM is insufficient.
- **`cache_dir`** — where downloaded weights are cached locally (defaults to `~/.cache/huggingface`).
- **`local_files_only`** — set `True` to force loading from the local cache only, useful in offline or airgapped environments.
- **`revision`** — pin a specific commit hash, tag, or branch of the model repo, so a re-run six months later doesn't silently pull a different checkpoint.
- **`token`** — a Hugging Face access token, required for **gated models** (checkpoints that require accepting a license on the Hub before download, e.g. many Llama releases).

6.2 Gated Models and Authentication

```python
from huggingface_hub import login
login(token="hf_...")  # read this from an environment variable, never hard-code it
```

```
import os
from huggingface_hub import login
login(token=os.environ["HF_TOKEN"])
```

- **Best Practice**

Always pin `revision` for anything beyond local experimentation. Model authors update repos in place – config changes, chat-template changes, even weight updates – and an unpinned revision breaks reproducibility silently.

Chapter 7

Chapter 8.7 — Text Generation

Decoding parameters control *how* a token is chosen from the model's output probability distribution. They do not change the model's weights or "knowledge" — they change how conservatively or creatively it samples.

7.1 Parameters

- **Greedy decoding** — always pick the single highest-probability token. Deterministic, but prone to repetition and blandness on longer generations.
- **Sampling** — draw the next token from the probability distribution rather than always taking the top one, introducing controlled randomness.
- **Temperature** — scales the logits before softmax. Below 1.0 sharpens the distribution (more deterministic, more repetitive); above 1.0 flattens it (more random, more diverse, higher risk of incoherence). `temperature=0` is equivalent to greedy decoding for most implementations.
- **Top-k** — restrict sampling to only the `k` highest-probability tokens at each step, discarding the long tail entirely.
- **Top-p (nucleus sampling)** — restrict sampling to the smallest set of tokens whose cumulative probability exceeds `p`. Adapts the candidate pool size per step, unlike the fixed pool size of top-k.
- **Repetition penalty** — down-weights tokens that already appeared in the generated sequence, reducing loops and verbatim repeats.
- **`max_new_tokens`** — hard cap on generated length; distinct from the model's total context window.
- **Stopping criteria** — stop sequences (specific strings/tokens) or custom logic that ends generation early, e.g. when the model emits an end-of-turn token or a user-defined delimiter.

```
output_ids = model.generate(
    **tokenizer(prompt, return_tensors="pt").to(model.device),
    max_new_tokens=300,
    do_sample=True,
    temperature=0.7,
    top_p=0.9,
    top_k=50,
    repetition_penalty=1.1,
    eos_token_id=tokenizer.eos_token_id,
)
print(tokenizer.decode(output_ids[0], skip_special_tokens=True))
```

7.1.1 LAB 2 — Compare Decoding Strategies

Using the same prompt, generate output under: (a) greedy decoding, (b) `temperature=0.3`, (c) `temperature=1.2`, (d) `top_p=0.5`. Change exactly one parameter per run. Record and compare the outputs — note where repetition, incoherence, or blandness appear.

Chapter 8

Chapter 8.8 — Decoding Strategies

Strategy	Deterministic	Diversity	Speed	Best Use
Greedy	Yes	None	Fastest	Deterministic tasks: extraction, classification, structured output
Beam search	Yes (per beam count)	Low–Medium	Slower (scales with beam width)	Translation, tasks with one "correct" phrasing, short constrained outputs
Temperature sampling	No	Tunable via temperature	Fast	Creative writing, brainstorming, varied phrasing
Top-k sampling	No	Bounded by k	Fast	General-purpose creative generation with a quality floor
Top-p (nucleus) sampling	No	Adapts to context confidence	Fast	Most production chat/creative use cases — the common default

- **Greedy** — use when you need reproducibility and the "best single answer," e.g. extracting a field value, classifying a label.
- **Beam search** — explores multiple candidate sequences in parallel and keeps the highest-scoring one; better than greedy for short, well-defined outputs (translation) but tends to produce generic, repetitive text on long open-ended generation and is rarely used for modern chat LLMs.
- **Temperature / top-k / top-p sampling** — the standard choice for conversational and creative generation; top-p is the most commonly used default in production chat APIs because it adapts to the model's confidence at each step rather than using a fixed cutoff.

• Interview Trap

"Just set temperature to 0 for factual accuracy" is a half-truth. Temperature 0 makes output deterministic and reduces creative drift, but it does nothing to prevent hallucination – a greedy-decoded answer can be a fully deterministic, perfectly confident wrong answer. Grounding (Chapter 8.34), not decoding strategy, is what controls factual accuracy.

★ Quick Revision Sheet

Chapters 8.1–8.8, in one screen:

- Pipeline: text → tokenizer → token IDs → embeddings → transformer blocks → logits → decoding → text.
- Base model completes text; instruction-tuned model follows tasks; chat model handles multi-turn roles.
- "Open weight" ≠ "open source" ≠ "source-available" – check the actual license per checkpoint.
- Model selection is a weighted, multi-factor decision – task quality on *your* eval set, context length, latency, hardware fit, license, cost, privacy.
- `torch_dtype` and `device_map` are the two parameters that most directly determine whether a model loading call succeeds or OOMs.
- Decoding parameters (temperature, top-k, top-p, repetition penalty) control randomness of token selection – they do not add or remove knowledge, and they do not fix hallucination.
- Top-p (nucleus) sampling is the most common production default; greedy is for deterministic/structured tasks; beam search is mostly out of favor for open-ended chat.

Chapter 9

Chapter 8.9 — Chat Models & Chat Templates

Chat models are trained on structured, multi-turn data with explicit roles. Feeding them raw concatenated strings instead of the format they were trained on is one of the most common — and most silent — sources of degraded output.

9.1 Roles

- **`system`** — instructions that set the assistant's behavior, persona, or constraints for the whole conversation. Not all models honor a `system` role equally strongly; check the model card.
- **`user`** — the human's message.
- **`assistant`** — the model's own prior turns, included so the model has conversational memory across a multi-turn exchange.

9.2 Why Manual Concatenation Fails

Every chat-tuned model is trained with specific **special tokens** wrapping each turn (e.g. turn-start/turn-end markers, role markers). These are model-specific and not documented consistently in prose — they're encoded in the model's `tokenizer_config.json` as a Jinja2 chat template. If you concatenate `"System: ...\nUser: ...\nAssistant:"` by hand, you are feeding the model a string that does not match its training distribution. Small models are especially sensitive to this and may ignore instructions, hallucinate additional turns, or fail to stop generating.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct")

messages = [
    {"role": "system", "content": "You are a concise technical assistant."},
    {"role": "user", "content": "What is a KV cache?"},
]

prompt = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
```

```
)
print(prompt)  # inspect the exact special-token structure this model expects
```

```
inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

output_ids = model.generate(inputs, max_new_tokens=200)
```

- **Common Mistake**

Manually writing `"<|user|> ... <|assistant|>"` from a blog post you read about a *different* model family. Special tokens are not standardized across model families – Llama, Qwen, Mistral, and Gemma each use different turn markers. Always call `apply_chat_template` rather than hard-coding token strings.

Chapter 10

Chapter 8.10 — Prompt Engineering

Prompt engineering is the practice of structuring your input to reliably get the output shape and quality you need, without changing the model.

10.1 Core Techniques

- **Clear instructions** — state the task explicitly rather than implying it.
- **Context** — give the model the background facts it needs; it cannot infer information you didn't provide.
- **Constraints** — length limits, tone, format, things to avoid.
- **Output format** — specify the exact shape you want (bullet list, JSON, a table) rather than hoping the model infers it.
- **Examples (few-shot)** — show 1–5 input/output pairs demonstrating the pattern you want.
- **Role specification** — framing the model's persona or expertise (`"You are a senior backend engineer reviewing this PR"`) shapes vocabulary and depth, though it does not grant the model real capabilities it lacks.
- **Decomposition** — break a complex task into explicit sub-steps rather than asking for the end result in one instruction.
- **Delimiters** — use clear markers (triple backticks, XML-style tags, ###) to separate instructions from the content being operated on, especially user-supplied or retrieved content.
- **Negative constraints** — explicitly state what *not* to do (`"Do not include a preamble"`) when the model's default behavior conflicts with what you need.

10.2 Before / After

Before:

```
Summarize this.
```

After:

```
Summarize the following support ticket in 2 sentences, written for a
non-technical manager. Do not include the customer's name or account ID.

Ticket:
"""
{ticket_text}
"""
```

- **Best Practice**

Put user-supplied or retrieved content inside explicit delimiters (triple quotes, XML tags) and refer to it by that delimiter in your instructions ("Summarize the ticket above between the triple quotes"). This reduces the chance that content inside the delimiter is misread as part of your instructions – and it's your first line of defense against prompt injection (Chapter 8.37).

Chapter 11

Chapter 8.11 — Prompt Design Patterns

Pattern	Description	When to use
Zero-shot	Task description only, no examples	Simple, well-understood tasks; strong instruction-following models
Few-shot	Task description + 1–5 labeled examples	Tasks with a specific output style/format the model needs to mimic
Structured prompt	Explicit sections (role, task, constraints, format)	Production prompts that need to be reliable and maintainable
Classification prompt	Ask for one label from a fixed, enumerated set	Categorization, routing, moderation
Extraction prompt	Ask for specific fields pulled from unstructured text	Turning documents/emails into structured records
Summarization prompt	Compress text while preserving key information	Long-document digestion, RAG context compression
Transformation prompt	Rewrite text into a different form (tone, language, format)	Style transfer, translation, format conversion
Reasoning-oriented prompt	Ask the model to work through a problem step by step, exposing only the final structured answer	Multi-step logic/arithmetic tasks — surface the conclusion, not necessarily every internal step, in production output
Grounded prompt	Instruct the model to answer using *only* supplied context	RAG, any task where hallucination risk must be minimized
Retrieval prompt	Combine a user query with retrieved documents and citation instructions	RAG answer generation (Chapter 8.27)

11.0.1 Classification Prompt Example

```
Classify the sentiment of the review below as exactly one of:
positive, negative, neutral.
Respond with only the label, nothing else.

Review: "{review_text}"
Label:
```

11.0.2 Extraction Prompt Example

```
Extract the following fields from the email below as JSON:
- sender_intent (one of: complaint, question, praise, other)
- product_mentioned (string or null)
- urgency (low, medium, high)

Email:
"""
{email_text}
"""
```

11.0.3 Grounded Prompt Example

```
Answer the question using ONLY the context below. If the context does
not contain enough information to answer, respond exactly with:
"I don't have enough information to answer that."

Context:
"""
{retrieved_context}
"""

Question: {user_question}
```

Chapter 12

Chapter 8.12 — Structured Outputs

Free-form text is not machine-readable. Production systems need the model's output to parse deterministically into a data structure.

```
User Input
 ↓
LLM
 ↓
Structured Output
 ↓
Validation
 ↓
Application Logic
```

12.1 Approaches

- **Prompted JSON** — instruct the model to output only JSON matching a described shape. Works reasonably with strong instruction-tuned models but is not guaranteed — the model can still emit invalid JSON, extra prose, or wrong types.
- **JSON Schema** — describe the exact required shape (field names, types, required vs. optional) so both the prompt and your validation logic reference the same contract.
- **Constrained decoding (concept)** — some serving stacks can mask the logits at each decoding step so only tokens consistent with a grammar/schema are ever sampled, making invalid output structurally impossible rather than merely discouraged. This is a serving-layer feature (varies by inference stack), not something you get for free by prompting alone.
- **Schema validation** — after generation, validate the output against the schema in code; never trust unvalidated model output as if it were guaranteed structured data.

12.1.1 Pydantic Preview

```
from pydantic import BaseModel, Field
from typing import Literal

class TicketExtraction(BaseModel):
    sender_intent: Literal["complaint", "question", "praise", "other"]
```

```
    product_mentioned: str | None = None
    urgency: Literal["low", "medium", "high"]

raw_output = llm_call(prompt)  # returns a string

try:
    parsed = TicketExtraction.model_validate_json(raw_output)
except Exception as e:
    # retry, log, or fall back — never pass unvalidated output downstream
    raise
```

Full Pydantic-based structured-output pipelines (with automatic retry-on-validation-failure and schema-to-prompt generation) are covered in depth in the production/agent engineering material — this chapter gives you the pattern, not the framework.

- **Common Mistake**

Calling `json.loads()` directly on raw model output with no try/except and no schema validation. Models occasionally wrap JSON in a markdown code fence, add a trailing sentence, or emit a subtly wrong type (a string where you expected an int). Always validate against a schema and handle the failure path explicitly.

Chapter 13

Chapter 8.13 — LLM API Engineering

Most production systems call a hosted LLM API rather than (or in addition to) running local inference. The engineering patterns are the same across providers even though request/response shapes differ slightly.

13.1 Core Patterns

```
import os
import anthropic

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=500,
    system="You are a concise technical writing assistant.",
    messages=[
        {"role": "user", "content": "Explain top-p sampling in two sentences."}
    ],
)
print(response.content[0].text)
```

- **Authentication** — API keys read from environment variables (`os.environ[...]`), never hard-coded in source. Use a `.env` file locally (excluded from version control) and a secrets manager in production.
- **Request construction** — model name, `messages` array with roles, `max_tokens`, and optional `system`, `temperature`, `top_p`.
- **Response parsing** — response objects nest the actual text inside a content structure that varies by provider; always parse defensively.
- **Retries** — transient network errors and `5xx` responses should be retried with backoff; `4xx` errors (bad request, invalid API key) should not be retried blindly.
- **Timeouts** — set an explicit request timeout; a hung request without one can block a worker indefinitely.
- **Rate limits** — providers return `429` responses when you exceed request-per-minute or token-per-minute limits; handle with exponential backoff and jitter.
- **Errors** — distinguish retryable errors (timeouts, `5xx`, `429`) from non-retryable ones (`401` auth failure, `400` malformed request) in your error-handling logic.

```python
import time
import random

def call_with_retry(fn, max_retries=5):
    for attempt in range(max_retries):
        try:
            return fn()
        except (TimeoutError, ConnectionError) as e:
            if attempt == max_retries - 1:
                raise
            sleep_time = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(sleep_time)
```

- **Best Practice**

Load API keys via `os.environ` and keep a `.env.example` file (with placeholder values, never real keys) in every project so collaborators know what environment variables they need to set. Add `.env` to `.gitignore` on day one, not after a key leaks.

Chapter 14

Chapter 8.14 — Streaming LLM Responses

Streaming returns tokens incrementally as they're generated, rather than waiting for the full response — critical for perceived latency in chat interfaces.

14.1 Concepts

- **Streaming tokens** — the server sends partial output as soon as each token (or small batch of tokens) is generated, instead of buffering the entire response.
- **Server-Sent Events (SSE)** — the common transport for streaming HTTP responses: a persistent connection where the server pushes discrete `data: ...` events as they become available.
- **Incremental output** — the client accumulates and displays each chunk as it arrives.
- **Async streaming** — using `async`/`await` so a server can handle many concurrent streaming requests without blocking a thread per connection.
- **Handling partial responses** — your client code must tolerate a connection dropping mid-stream, and must correctly reassemble chunks that may split mid-word depending on the tokenization boundary.

```
import anthropic

client = anthropic.Anthropic()

with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=500,
    messages=[{"role": "user", "content": "Write a haiku about tokenization."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

14.1.1 Minimal FastAPI Streaming Endpoint

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import anthropic

app = FastAPI()
```

```python
client = anthropic.Anthropic()

def token_generator(prompt: str):
    with client.messages.stream(
        model="claude-sonnet-4-6",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        for text in stream.text_stream:
            yield text

@app.get("/generate")
def generate(prompt: str):
    return StreamingResponse(token_generator(prompt), media_type="text/plain")
```

★ Quick Revision Sheet

Chapters 8.9–8.14, in one screen:

- Always use `apply_chat_template` for chat models – manual concatenation targets the wrong special-token format.
- Prompt engineering levers: instructions, context, constraints, format, few-shot examples, role, decomposition, delimiters, negative constraints.
- Match the prompt pattern to the task: classification → enumerated label; extraction → schema fields; RAG → grounded prompt with an explicit "don't know" instruction.
- Structured output = prompted/constrained JSON generation *plus* schema validation in code – never trust unvalidated model output.
- API engineering = env-var auth, explicit timeouts, retry only on retryable errors (5xx/429/timeouts), backoff with jitter.
- Streaming = incremental token delivery over SSE/async, mainly a perceived-latency optimization for interactive UIs.

Chapter 15

Chapter 8.15 — Embeddings

An **embedding model** maps text to a fixed-length dense vector that captures semantic meaning — texts with similar meaning produce vectors that are close together in that vector space. This is the foundation of semantic search, retrieval, and RAG.

15.1 Core Concepts

- **Embedding vector** — a fixed-length array of floats (e.g. 384, 768, 1024, 3072 dimensions) representing a piece of text.
- **Dimensions** — the vector's length; higher-dimensional embeddings can encode more nuance but cost more to store and compare.
- **Semantic meaning** — embeddings are trained so that geometric closeness (cosine similarity, dot product, or Euclidean distance) approximates human-judged semantic similarity.
- **Similarity** — the numeric measure of closeness between two embeddings (Chapter 8.17 covers cosine similarity specifically).
- **Normalization** — scaling a vector to unit length (L2 norm = 1). Many embedding models are trained expecting normalized vectors, and normalization makes cosine similarity and dot-product-based search equivalent.
- **Dense vectors** — every dimension carries information (as opposed to sparse representations like TF-IDF, where most entries are zero).

15.2 Embedding Model ≠ Generative LLM

This distinction matters and is a common source of confusion:

	Embedding model	Generative LLM
Output	One fixed-length vector	A sequence of generated tokens
Purpose	Represent meaning for comparison/search	Produce new text
Typical size	Much smaller (millions–low billions of params)	Larger (billions–hundreds of billions)
Used for	Semantic search, retrieval, clustering, dedup	Chat, generation, reasoning, summarization

You cannot use a chat LLM's hidden states as a drop-in embedding model without specific adaptation — embedding

models are trained with a *contrastive* objective specifically to make similar texts land close together in vector space; generative LLMs are trained with a next-token-prediction objective that doesn't guarantee that property.

15.3 Practical Example — Sentence Transformers

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

texts = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "The stock market fell sharply today.",
]

embeddings = model.encode(texts, normalize_embeddings=True)
print(embeddings.shape)  # (3, 384)
```

15.4 Practical Example — Raw Hugging Face

```
from transformers import AutoTokenizer, AutoModel
import torch
import torch.nn.functional as F

model_id = "sentence-transformers/all-MiniLM-L6-v2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModel.from_pretrained(model_id)

def mean_pooling(model_output, attention_mask):
    token_embeddings = model_output[0]
    mask = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    return torch.sum(token_embeddings * mask, 1) / torch.clamp(mask.sum(1), min=1e-9)

inputs = tokenizer(["The cat sat on the mat."], padding=True, truncation=True, return_tensors="pt")
with torch.no_grad():
    output = model(**inputs)
embedding = F.normalize(mean_pooling(output, inputs["attention_mask"]), p=2, dim=1)
```

Chapter 16

Chapter 8.16 — Embedding Model Selection

Category	Example use case	Tradeoff
General-purpose	English-only knowledge base search	Good default, smaller footprint
Multilingual	Cross-language or non-English document search	Usually larger, slightly lower per-language quality than a dedicated monolingual model
Small (e.g. ~50–100M params)	Latency-sensitive, high-QPS, or edge deployment	Faster and cheaper, some retrieval-quality ceiling
Large (e.g. ~1B+ params)	Quality-sensitive retrieval where latency budget allows	Higher retrieval quality, more compute/memory per query
Domain-specific (legal, medical, code)	Specialized corpora where general embeddings under-perform	Best in-domain quality, worst out-of-domain generalization

16.1 What to Weigh

- **Dimensions** — larger dimension count generally correlates with more nuance but increases storage and search cost (a 1M-document index at 1536 dimensions in float32 is roughly 6GB just for vectors, before any index overhead).
- **Speed** — embedding throughput matters at ingestion time for large corpora, and at query time for latency-sensitive applications.
- **Memory** — vector storage scales linearly with document count × dimensions × bytes-per-float.
- **Retrieval quality** — always validate on your own document set and query set (Chapter 8.32); published benchmark rankings (e.g. general leaderboards) are a starting point, not a guarantee for your domain.
- **Language support** — general-purpose English models often underperform badly on non-English text; verify explicitly if your corpus is multilingual.
- **Deployment constraints** — a model that's free to call via a hosted API has different operational tradeoffs than one you self-host.

- **Common Mistake**

Picking an embedding model purely off a public leaderboard rank without testing on your actual documents. Leaderboards measure aggregate performance across benchmark datasets that may look nothing like your domain (e.g. internal legal contracts, medical notes, code). Always run a small retrieval-quality test (Chapter 8.32) on a sample of your own data before committing to a model for a production index.

Chapter 17

Chapter 8.17 — Cosine Similarity for Semantic Search

```
Query
 ↓
Embedding
 ↓
Document Embeddings
 ↓
Cosine Similarity
 ↓
Ranking
 ↓
Top-k Documents
```

Cosine similarity measures the cosine of the angle between two vectors — it captures *orientation* similarity, independent of vector magnitude, which makes it robust to differences in text length between the query and documents.

17.1 From Scratch (NumPy)

```
import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

query_vec = np.array([0.12, 0.87, -0.33])
doc_vec = np.array([0.10, 0.90, -0.30])
print(cosine_similarity(query_vec, doc_vec))
```

17.2 With scikit-learn

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

```
doc_matrix = np.array(document_embeddings)   # shape (n_docs, dim)
query_matrix = np.array([query_embedding])   # shape (1, dim)

scores = cosine_similarity(query_matrix, doc_matrix)[0]
top_k_idx = scores.argsort()[::-1][:5]
```

17.3 With Sentence Transformers

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

query_embedding = model.encode("How does top-p sampling work?", normalize_embeddings=True)
doc_embeddings = model.encode(documents, normalize_embeddings=True)

scores = util.cos_sim(query_embedding, doc_embeddings)[0]
top_k = scores.topk(5)
```

- **Best Practice**

Normalize embeddings once at index-build time (not per query) whenever your embedding model and search library support it. With normalized vectors, cosine similarity and dot product produce identical rankings, and dot product is cheaper to compute – this is why FAISS's `IndexFlatIP` on normalized vectors is a common way to get "cosine similarity search" efficiently.

Chapter 18

Chapter 8.18 — Vector Database Fundamentals

A vector database stores embeddings alongside metadata and provides efficient similarity search over potentially millions of vectors — far faster than brute-force comparison against every stored vector.

18.1 Core Concepts

- **Vector storage** — persisting embeddings (and their source metadata) durably, not just in memory.
- **Metadata** — arbitrary key/value data attached to each vector (source document, page number, timestamp, access-control tags) used for filtering and citation.
- **Indexing** — building a data structure (e.g. HNSW graphs, IVF clusters) that makes approximate nearest-neighbor search sub-linear in the number of vectors, trading a small amount of recall for large speed gains at scale.
- **Similarity search** — the core query operation: given a query vector, return the most similar stored vectors.
- **Top-k retrieval** — returning the `k` highest-scoring matches rather than a full ranked list.
- **Filtering** — restricting search to vectors whose metadata matches a condition (e.g. `document_type == "policy"` or `access_level <= user_level`), either before or after the similarity computation depending on the engine.
- **Persistence** — durability across restarts; distinguishes a production vector database from an in-memory index you rebuild every run.

18.2 Comparison

Database	Local	Cloud	Filtering	Scale	Complexity
FAISS	Yes (library, not a server)	No (self-managed)	Manual (you implement it)	Very high (billions of vectors, with the right index type)	Low to use, higher to operate at scale
Chroma	Yes	Limited (hosted option exists)	Built-in metadata filtering	Small–medium	Low — easiest to start with
Qdrant	Yes	Yes (managed cloud)	Rich built-in filtering	Medium–high	Medium

Database	Local	Cloud	Filtering	Scale	Complexity
Weaviate	Yes	Yes (managed cloud)	Rich built-in filtering, hybrid search	Medium–high	Medium
Milvus	Yes	Yes (managed cloud, Zilliz)	Rich built-in filtering	Very high	Higher — built for large-scale deployment

There is no universal "best" choice. FAISS is a library, not a database — you get raw speed and full control but build persistence, filtering, and metadata handling yourself. Chroma, Qdrant, Weaviate, and Milvus are full databases with metadata, filtering, and persistence built in, differing mainly in scale target and operational complexity.

Chapter 19

Chapter 8.19 — FAISS

FAISS (Facebook AI Similarity Search) is a library for efficient vector similarity search — the workhorse for understanding what a vector database is doing underneath its API.

```
pip install faiss-cpu   # or faiss-gpu for CUDA-accelerated search
```

19.1 Core Index Types

- **`IndexFlatL2`** — brute-force search using Euclidean (L2) distance. Exact, simple, slow at large scale (linear in the number of vectors).
- **`IndexFlatIP`** — brute-force search using inner product (dot product). On *normalized* vectors, ranking by inner product is equivalent to ranking by cosine similarity — this is the standard way to do "cosine similarity search" in FAISS.

```
import faiss
import numpy as np

dimension = 384
index = faiss.IndexFlatIP(dimension)  # inner product on normalized vectors == cosine similarity

# vectors: shape (n_docs, dimension), already L2-normalized
vectors = np.random.rand(1000, dimension).astype("float32")
faiss.normalize_L2(vectors)
index.add(vectors)

query = np.random.rand(1, dimension).astype("float32")
faiss.normalize_L2(query)

k = 5
scores, indices = index.search(query, k)
print(indices, scores)
```

19.2 Persistence

```
faiss.write_index(index, "my_index.faiss")
loaded_index = faiss.read_index("my_index.faiss")
```

19.3 L2 Distance vs. Inner Product vs. Cosine Similarity

- **L2 distance** measures straight-line distance between two vectors — smaller means more similar.
- **Inner product (dot product)** measures both magnitude and orientation — larger means more similar, but is sensitive to vector length.
- On **L2-normalized** vectors (all vectors scaled to unit length), inner product and cosine similarity produce identical rankings, and L2 distance becomes a monotonic function of cosine similarity too. This is why normalization (Chapter 8.15) is the standard practice before indexing.

19.3.1 LAB 4 — Implement FAISS Search

Embed a set of ~200 short documents with a Sentence Transformers model, normalize them, build an `IndexFlatIP` index, persist it to disk, reload it, and run 5 test queries. Record retrieval results and latency.

Chapter 20

Chapter 8.20 — Vector Database Project

Build a complete, minimal semantic-search application end to end.

20.1 Pipeline

```
Dataset (documents) → Embed → Build FAISS Index → Store Metadata → Query → Retrieve Top-k → Rank → Evaluate
```

```python
import faiss
import numpy as np
import json
from sentence_transformers import SentenceTransformer

# 1. Dataset — a list of {"id": ..., "text": ..., "source": ...} records
documents = json.load(open("documents.json"))

# 2. Embed
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
texts = [d["text"] for d in documents]
embeddings = model.encode(texts, normalize_embeddings=True, show_progress_bar=True)
embeddings = np.array(embeddings).astype("float32")

# 3. Build FAISS index
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(embeddings)

# 4. Store metadata (aligned by row index with the FAISS index)
metadata = [{"id": d["id"], "source": d["source"], "text": d["text"]} for d in documents]

# 5. Query
def search(query: str, k: int = 5):
    query_vec = model.encode([query], normalize_embeddings=True).astype("float32")
    scores, indices = index.search(query_vec, k)
    return [
        {"score": float(scores[0][i]), **metadata[idx]}
```

```
        for i, idx in enumerate(indices[0])
    ]

results = search("How do I reset my password?")
for r in results:
    print(r["score"], r["source"], r["text"][:80])
```

20.2 Environment Specification

```
Python: 3.10+
sentence-transformers>=2.7
faiss-cpu>=1.8
numpy>=1.26
CPU: any modern x86_64/ARM, no GPU required at this scale
RAM: ~2GB for a 10K-document corpus with a 384-dim model
Expected latency: <50ms per query at 10K documents (IndexFlatIP, CPU)
```

20.3 Evaluation

At minimum, hand-label 10–20 (query, expected top document) pairs and check whether the expected document appears in the top-k results (Recall@k) — this is the smallest useful evaluation and a preview of Chapter 8.32's fuller treatment.

★ Quick Revision Sheet

Chapters 8.15–8.20, in one screen:

- Embedding models produce one fixed-length vector per text, trained with a contrastive objective so similar meaning → close vectors – this is a different model class from a generative LLM.
- Always validate embedding model choice against your own corpus and queries, not a generic leaderboard.
- Cosine similarity = orientation similarity, robust to text-length differences; normalize vectors once at index time so inner product becomes equivalent to cosine similarity.
- A vector database = storage + indexing + similarity search + metadata filtering + persistence. FAISS gives you the first three as a fast library; Chroma/Qdrant/Weaviate/Milvus add metadata, filtering, and persistence out of the box.
- `IndexFlatIP` on L2-normalized vectors is the standard FAISS pattern for cosine-similarity search.
- Minimum viable evaluation: hand-label a handful of (query, expected document) pairs and measure Recall@k before trusting any retrieval pipeline.

Chapter 21

Chapter 8.21 — Document Ingestion

Before anything can be embedded or retrieved, real-world documents have to be turned into clean text. This is unglamorous and is also where most production RAG systems quietly lose quality.

21.1 Extraction vs. Retrieval

Extraction is pulling raw text out of a source file format. **Retrieval** is finding the relevant *pieces* of that text for a given query. Extraction happens once per document at ingestion time; retrieval happens once per query. Bad extraction (garbled tables, missing sections, broken encoding) silently caps the ceiling on retrieval quality no matter how good your retrieval pipeline is.

21.2 Format-by-Format Notes

- **TXT** — trivial; watch for encoding issues (always specify `encoding="utf-8"` explicitly, don't rely on system defaults).
- **PDF** — the hardest common case. Text-based PDFs extract cleanly with libraries like `pypdf` or `pdfplumber`; scanned/image PDFs require OCR first. Tables and multi-column layouts often extract out of reading order — verify extracted output against the source visually for any document with complex layout.
- **DOCX** — use `python-docx` to walk paragraphs, tables, and headers/footers explicitly rather than treating the file as flat text.
- **HTML** — strip navigation, ads, and boilerplate before extraction (e.g. with `BeautifulSoup` plus a content-extraction heuristic) — raw HTML-to-text conversion pulls in a lot of non-content noise.
- **Markdown** — closest to already being clean text; still worth stripping raw syntax (`#`, `**`, links) if you want the chunk text itself to read naturally, or preserving it if headers are useful chunk boundaries (Chapter 8.22).
- **CSV** — usually ingested as structured records rather than free text; decide per-row vs. per-table chunking based on whether a single row is a meaningful retrieval unit.
- **JSON** — similar to CSV: decide what unit of the JSON structure maps to one retrievable "document."

```
from pathlib import Path
import pypdf

def extract_pdf_text(path: Path) -> list[dict]:
    reader = pypdf.PdfReader(str(path))
    pages = []
    for i, page in enumerate(reader.pages):
```

```
        text = page.extract_text() or ""
        pages.append({"page": i + 1, "text": text, "source": path.name})
    return pages
```

- **Common Mistake**

Running a scanned PDF through a text-extraction library and getting an empty string per page, then silently indexing zero content for that document. Always assert non-trivial extracted text length per document during ingestion and log/flag documents that fall through — a scanned document needs OCR (see the PDF skill/tooling for OCR workflows), not the same extraction path as a text-based PDF.

Chapter 22

Chapter 8.22 — Document Chunking

Chunking is one of the most important decisions in a RAG pipeline — more so, in practice, than which embedding model or which LLM you choose. A chunk that's too large dilutes the specific fact a query needs with irrelevant surrounding text; a chunk that's too small loses the surrounding context needed to interpret it correctly.

22.1 Strategies

- **Fixed-size chunks** — split every `N` characters, with no awareness of sentence or paragraph boundaries. Simple, fast, but frequently cuts sentences (and facts) in half.
- **Token-based chunks** — split by token count (using the target embedding model's tokenizer) rather than characters, so chunk size maps directly to what the model actually "sees" and to embedding-model input limits.
- **Sentence-based chunks** — split on sentence boundaries, then group sentences up to a target size. Preserves grammatical units.
- **Paragraph-based chunks** — use natural paragraph breaks as the unit; works well for well-structured source documents, poorly for documents with very long or very short paragraphs.
- **Recursive splitting** — try to split on the largest natural boundary first (section → paragraph → sentence → word), falling back to a smaller boundary type only when a piece still exceeds the target size. This is the most commonly used general-purpose strategy in practice because it adapts to document structure while still guaranteeing a size ceiling.
- **Semantic chunking (concept)** — split based on detected shifts in topic/meaning between sentences (e.g. by embedding consecutive sentences and cutting where similarity drops), rather than a fixed size or fixed structural boundary. More adaptive, more expensive to compute, and its benefit over recursive splitting is workload-dependent — validate it against your own eval set rather than assuming it's strictly better.
- **Overlap** — including the last `N` tokens/characters of one chunk at the start of the next, so information near a chunk boundary isn't orphaned from its context in *either* chunk.
- **Metadata** — every chunk should carry its source document, position, and any structural context (section heading) needed to interpret it standalone (Chapter 8.24).

22.1.1 Recursive Splitting Example

```python
def recursive_split(text: str, max_chars: int = 800, overlap: int = 100) -> list[str]:
    separators = ["\n\n", "\n", ". ", " "]
    return _split(text, separators, max_chars, overlap)

def _split(text: str, separators: list[str], max_chars: int, overlap: int) -> list[str]:
```

```
    if len(text) <= max_chars:
        return [text]
    if not separators:
        # last resort: hard character split
        return [text[i:i + max_chars] for i in range(0, len(text), max_chars - overlap)]

    sep = separators[0]
    pieces = text.split(sep)
    chunks, current = [], ""
    for piece in pieces:
        candidate = current + sep + piece if current else piece
        if len(candidate) <= max_chars:
            current = candidate
        else:
            if current:
                chunks.append(current)
            current = piece
    if current:
        chunks.append(current)

    # re-split any chunk still too large using the next separator
    result = []
    for chunk in chunks:
        if len(chunk) > max_chars:
            result.extend(_split(chunk, separators[1:], max_chars, overlap))
        else:
            result.append(chunk)

    # apply overlap between consecutive chunks
    overlapped = []
    for i, chunk in enumerate(result):
        if i == 0:
            overlapped.append(chunk)
        else:
            prefix = result[i - 1][-overlap:]
            overlapped.append(prefix + chunk)
    return overlapped
```

- **Best Practice**

Default to recursive splitting with a size ceiling of roughly 300-800 tokens and an overlap of 10-20% of chunk size as a starting point, then tune both against your own retrieval-quality evaluation (Chapter 8.32) rather than a rule of thumb from a blog post. The right chunk size is a function of your document structure and your typical query specificity, not a universal constant.

Chapter 23

Chapter 8.23 — Chunking Experiment Lab

23.0.1 LAB 5 — Experiment With Chunk Sizes

Using the same source document and the same 10–15 test queries, build four separate indexes:

1. Small chunks (~150 tokens, no overlap)
2. Medium chunks (~400 tokens, 15% overlap)
3. Large chunks (~1000 tokens, 15% overlap)
4. Medium chunks with different overlap (~400 tokens, 40% overlap)

For each configuration, measure and record:

Metric	Small	Medium (15%)	Large	Medium (40%)
Retrieval precision (manual judgment on top-3)				
Answer quality (manual judgment, 1-5)				
Average retrieved context size (tokens)				
End-to-end query latency				

There is no universally correct answer here — the point of the lab is to observe, on your own document set, that chunk size measurably changes both retrieval precision and downstream answer quality, and to develop the habit of testing this instead of guessing.

Chapter 24

Chapter 8.24 — Metadata

Metadata is structured information attached to each chunk, separate from the chunk's embedded text, used for filtering, citation, debugging, and access control.

```
{
  "source": "employee_handbook_v3.pdf",
  "page": 4,
  "section": "Remote Work Policy",
  "document_id": "doc_00931",
  "chunk_id": "doc_00931_chunk_012",
  "timestamp": "2026-01-15T00:00:00Z",
  "author": "HR Policy Team"
}
```

24.1 Why Metadata Matters

- **Filtering** — restrict search to a subset of documents (by type, date range, department) before or alongside the similarity search, rather than searching the entire corpus for every query.
- **Citations** — the `source` and `page`/`section` fields are exactly what you display back to the user as "where this answer came from" (Chapter 8.29).
- **Debugging** — when retrieval returns a wrong or surprising result, metadata lets you trace it back to the exact source document and location instead of staring at an opaque chunk of text.
- **Access control** — tagging chunks with permission scopes lets you enforce authorization at the retrieval layer (Chapter 8.38), which is essential when a corpus contains documents with different sensitivity levels.
- **Provenance** — knowing when a document was ingested and from what version helps you detect and handle stale or superseded content (Chapter 8.31).

```
chunk_record = {
    "chunk_id": f"{document_id}_chunk_{i:03d}",
    "document_id": document_id,
    "text": chunk_text,
    "source": filename,
    "page": page_number,
    "section": current_section_heading,
    "ingested_at": ingestion_timestamp,
}
```

Chapter 25

Chapter 8.25 — Retrieval

Retrieval is the step that turns a query into a ranked set of candidate chunks to hand to the LLM.

25.1 Retrieval Strategies

- **Dense retrieval** — embedding-based similarity search (Chapters 8.15–8.19). Strong at capturing semantic/paraphrase similarity; can miss exact keyword/entity matches that don't carry strong semantic signal.
- **Sparse retrieval** — keyword/term-frequency-based search (classically TF-IDF or BM25). Strong at exact matches (product codes, names, rare technical terms); weak at paraphrase and semantic similarity.
- **Keyword retrieval** — a simpler subset of sparse retrieval: literal term matching, sometimes with basic stemming/synonym expansion.
- **Hybrid retrieval** — combining dense and sparse signals (e.g. weighted score fusion, or reciprocal rank fusion) to get both semantic recall and exact-match precision.
- **Top-k** — how many candidates to retrieve per query; larger `k` increases recall but also increases context size and noise fed to the LLM.
- **Metadata filtering** — narrowing the candidate pool by structured criteria before or during similarity search (Chapter 8.24).

25.2 BM25 vs. Embeddings

	BM25 (sparse)	Embeddings (dense)
Matches	Exact/near-exact term overlap	Semantic/paraphrase similarity
Strength	Rare terms, IDs, exact names, acronyms	Conceptually related queries with different wording
Weakness	Misses paraphrases and synonyms entirely	Can miss exact-match precision on rare/technical terms
Cost	Cheap, no model inference needed	Requires an embedding model call per query and per document

25.3 Hybrid Search Pattern

```python
def hybrid_search(query, k=10, alpha=0.5):
    dense_results = dense_search(query, k=k * 2)   # embedding similarity
    sparse_results = bm25_search(query, k=k * 2)    # keyword score

    combined_scores = {}
    for rank, doc_id in enumerate(dense_results):
        combined_scores[doc_id] = combined_scores.get(doc_id, 0) + alpha * (1 / (rank + 1))
    for rank, doc_id in enumerate(sparse_results):
        combined_scores[doc_id] = combined_scores.get(doc_id, 0) + (1 - alpha) * (1 / (rank + 1))

    ranked = sorted(combined_scores.items(), key=lambda x: x[1], reverse=True)
    return [doc_id for doc_id, _ in ranked[:k]]
```

This is **reciprocal rank fusion** — a simple, robust way to combine two differently-scaled ranking signals without needing to calibrate their raw scores against each other.

• Interview Trap

”We use embeddings, so we get semantic search — we don’t need keyword search too.” This misses that dense retrieval reliably underperforms on queries containing exact identifiers (order numbers, product SKUs, error codes, proper nouns the embedding model saw rarely during training). Production RAG systems handling any structured or technical corpus commonly need hybrid retrieval, not dense-only.

Chapter 26

Chapter 8.26 — Reranking

Retrieval and reranking are two different jobs. A retriever (dense/sparse/hybrid) needs to be fast enough to search the *entire* corpus; a reranker only needs to be accurate over a small candidate set, which lets it afford a much more expensive, higher-quality relevance model.

```
Query
 ↓
Retriever
 ↓
Top 20
 ↓
Reranker
 ↓
Top 5
 ↓
LLM
```

26.1 Concepts

- **Initial retrieval** — cast a wide net (e.g. top 20–50) using a fast retriever, prioritizing recall over precision.
- **Candidate generation** — the output of initial retrieval: a manageable set of plausibly-relevant chunks.
- **Reranking** — re-score each candidate with a more accurate (and more expensive) relevance model, then keep only the top few.
- **Cross-encoder** — unlike a bi-encoder (which embeds the query and document separately, as in Chapter 8.15's dense retrieval), a cross-encoder takes the query and document *together* as joint input and outputs a single relevance score. This lets it model query-document interaction directly, at the cost of not being pre-computable/indexable — it must run at query time on every candidate pair, which is why it's used only on a small shortlist, not the whole corpus.
- **Relevance scoring** — the cross-encoder's output score, used purely to re-rank the shortlist, not to build any persistent index.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

candidates = retrieve_top_k(query, k=20)  # dense/hybrid retrieval, from Chapter 8.25
```

```
pairs = [[query, c["text"]] for c in candidates]
scores = reranker.predict(pairs)

reranked = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
top_5 = [c for c, s in reranked[:5]]
```

26.2 Why Retrieve-More-Then-Rerank-Fewer Improves Quality

A fast bi-encoder retriever optimizes for scalability across the whole corpus and can rank a genuinely relevant document at position 15 instead of position 2, because it never lets the query and document interact directly. Retrieving a wider top-20–50 candidate pool gives the reranker a good chance of *containing* the truly relevant documents; the cross-encoder's more accurate, interaction-aware scoring then correctly promotes them into the final top-5 that actually reaches the LLM.

26.2.1 LAB 8 — Add Reranking

Take the retrieval pipeline from Chapter 8.20's project. Retrieve the top 20 candidates per query, apply a cross-encoder reranker, and keep the top 5. Compare the top-5 results with and without reranking on your existing evaluation queries (Chapter 8.23) — record whether reranking changes which documents land in the final top-5.

★ Quick Revision Sheet

Chapters 8.21–8.26, in one screen:

- Extraction (raw text out of a file) happens once per document; retrieval (finding relevant pieces) happens once per query – bad extraction caps retrieval quality no matter what you do downstream.
- Chunking is arguably the highest-leverage RAG decision. Recursive splitting (structure-aware, size-capped) is the standard general-purpose default; always tune size/overlap against your own eval set.
- Every chunk needs metadata (source, position, section) for filtering, citation, debugging, and access control – it's not optional plumbing.
- Dense retrieval = semantic/paraphrase strength; sparse (BM25/keyword) retrieval = exact-match strength; hybrid combines both, commonly via reciprocal rank fusion.
- Reranking = retrieve a wide candidate pool cheaply (bi-encoder), then re-score a small shortlist expensively and accurately (cross-encoder) before it reaches the LLM.

Chapter 27

Chapter 8.27 — RAG Fundamentals

Retrieval-Augmented Generation combines everything from Chapters 8.15–8.26 into one pipeline: retrieve relevant knowledge at query time and inject it into the LLM's context, instead of relying purely on what the model memorized during training.

```
Documents
 ↓
Load
 ↓
Clean
 ↓
Chunk
 ↓
Embed
 ↓
Index
 ↓
Retrieve
 ↓
Rerank
 ↓
Context
 ↓
LLM
 ↓
Answer
```

27.1 Stage by Stage

- **Load** — read source documents from their storage location (Chapter 8.21).
- **Clean** — strip boilerplate, fix encoding, normalize whitespace.
- **Chunk** — split into retrievable units (Chapter 8.22).
- **Embed** — convert each chunk into a vector (Chapter 8.15).
- **Index** — store vectors + metadata in a searchable structure (Chapters 8.18–8.19).
- **Retrieve** — given a query, fetch the most relevant chunks (Chapter 8.25).

- **Rerank** (optional but recommended beyond small corpora) — re-score and narrow the candidate set (Chapter 8.26).
- **Context** — assemble the final chunks into a prompt-ready block, respecting token budget (Chapter 8.35).
- **LLM** — generate an answer conditioned on the assembled context.
- **Answer** — the final output, ideally with citations (Chapter 8.29).

The first six stages happen at **ingestion time** (once per document, or whenever documents change); the last four happen at **query time** (once per user question). Conflating these two timelines is a common source of confused system design — ingestion latency and query latency have completely different budgets and failure modes.

Chapter 28

Chapter 8.28 — Basic RAG Implementation

Build the full pipeline in plain Python, with no framework, so the mechanics are visible end to end. Frameworks (Part 11) are abstractions *over* this — understanding this first makes those abstractions legible instead of magical.

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
import anthropic
import os

embed_model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
llm_client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# --- Ingestion (once) ---

def build_index(chunks: list[dict]):
    texts = [c["text"] for c in chunks]
    embeddings = embed_model.encode(texts, normalize_embeddings=True).astype("float32")
    index = faiss.IndexFlatIP(embeddings.shape[1])
    index.add(embeddings)
    return index, chunks  # chunks acts as our metadata store, aligned by row index

# --- Query time ---

def retrieve(query: str, index, chunks: list[dict], k: int = 5) -> list[dict]:
    query_vec = embed_model.encode([query], normalize_embeddings=True).astype("float32")
    scores, indices = index.search(query_vec, k)
    return [
        {**chunks[idx], "score": float(scores[0][i])}
        for i, idx in enumerate(indices[0])
    ]

def build_prompt(query: str, retrieved_chunks: list[dict]) -> str:
    context = "\n\n".join(
        f"[Source: {c['source']}]\n{c['text']}" for c in retrieved_chunks
    )
```

```
    return f"""Answer the question using ONLY the context below. If the context
does not contain enough information, say so explicitly.

Context:
\"\"\"
{context}
\"\"\"

Question: {query}
Answer:"""

def answer(query: str, index, chunks: list[dict]) -> str:
    retrieved = retrieve(query, index, chunks, k=5)
    prompt = build_prompt(query, retrieved)
    response = llm_client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.content[0].text

# --- Usage ---

index, chunks = build_index(my_chunks)
print(answer("What is our remote work policy?", index, chunks))
```

28.0.1 LAB 6 — Build RAG Without LangChain

Using the pipeline above and a set of at least 20 documents, implement the full ingestion → retrieval → generation flow yourself. Test with 10 questions and manually judge whether each answer is grounded in the retrieved context.

Chapter 29

Chapter 8.29 — RAG with Citations

An ungrounded-looking answer erodes trust even when it's correct. Source-aware RAG makes provenance visible to the user.

```
Answer

Sources:
[1] document.pdf, page 4
[2] handbook.md, section 2.3
```

```
def build_prompt_with_citations(query: str, retrieved_chunks: list[dict]) -> str:
    context_blocks = []
    for i, c in enumerate(retrieved_chunks, start=1):
        context_blocks.append(f"[{i}] Source: {c['source']}, page {c.get('page', 'N/A')}\n{c['text']}")
    context = "\n\n".join(context_blocks)

    return f"""Answer the question using ONLY the numbered sources below.
After your answer, list the sources you actually used, citing them by
their bracketed number, in this exact format:

Sources:
[N] <source name>, <page or section>

If the sources do not contain enough information, say so and cite no sources.

Sources:
\"\"\"
{context}
\"\"\"

Question: {query}
Answer:"""
```

29.1 Citation Mapping

The key engineering discipline here: never let the model invent a citation. Map every citation the model outputs back to a real, retrieved chunk's metadata in your own code — if the model cites `[3]` but only 2 chunks were retrieved, that's a signal to flag or discard the citation, not to trust it.

```python
def validate_citations(answer_text: str, retrieved_chunks: list[dict]) -> bool:
    import re
    cited_numbers = {int(n) for n in re.findall(r"\[(\d+)\]", answer_text)}
    valid_numbers = set(range(1, len(retrieved_chunks) + 1))
    return cited_numbers.issubset(valid_numbers)
```

- **Best Practice**

Number sources explicitly in the prompt and require the model to cite by that number, then validate the cited numbers against the actual retrieved set in code. This is far more reliable than asking the model to reproduce filenames or page numbers from memory, which it can get subtly wrong even when grounded overall.

Chapter 30

Chapter 8.30 — Advanced RAG Patterns

The following are patterns worth knowing conceptually; this chapter distinguishes what's implemented in this Part versus what's concept-only (deeper implementation of some of these — especially agentic multi-step retrieval — belongs in Part 11).

Implemented as working code in this Part: hybrid search (Chapter 8.25), reranking (Chapter 8.26), metadata filtering (Chapter 8.24, 8.38).

Concept-only in this Part:

- **Query rewriting** — using an LLM call to reformulate a user's raw query into a better search query before retrieval (expanding abbreviations, resolving ambiguous pronouns from conversation history, splitting a compound question).
- **Multi-query retrieval** — generating several reformulated variants of the same query, retrieving for each, and merging/deduplicating the results — improves recall for ambiguous queries at the cost of extra retrieval calls and an extra LLM call.
- **Hypothetical Document Embeddings (HyDE)** — using an LLM to generate a hypothetical *answer* to the query first, then embedding that hypothetical answer (rather than the raw query) for retrieval, on the theory that an answer's embedding is closer to relevant documents' embeddings than a short question's embedding is.
- **Contextual compression** — after retrieval, using an LLM (or a smaller extraction model) to strip irrelevant sentences out of each retrieved chunk before it reaches the final generation call, reducing noise and token cost.
- **Parent-child retrieval** — indexing small chunks for precise retrieval matching, but returning the larger "parent" section they belong to as context, combining retrieval precision with generation-time completeness.
- **Multi-vector retrieval** — indexing multiple representations per document (e.g. a summary vector and several chunk vectors) so a query can match at whichever granularity is most relevant.
- **Hierarchical retrieval** — retrieving at a coarse level first (which document/section is relevant) and then at a fine level within that scope, useful for very large corpora where flat top-k search over everything becomes noisy.

- **Common Mistake**

Reaching for HyDE, multi-query retrieval, or contextual compression as a first fix when retrieval quality is poor. In practice, chunking strategy (Chapter 8.22) and basic hybrid search (Chapter 8.25) fix the large majority of retrieval-quality problems. Advanced patterns add real latency and cost – validate that a simpler fix genuinely isn't enough (Chapter 8.32's evaluation) before reaching for them.

Chapter 31

Chapter 8.31 — RAG Failure Modes

31.1 Common Failure Modes

- **Bad chunking** — facts split across chunk boundaries, or chunks too large to be specific.
- **Bad embeddings** — an embedding model mismatched to the domain or language of the corpus.
- **Irrelevant retrieval** — the retriever returns chunks that are topically similar but don't actually answer the query.
- **Insufficient retrieval** — `k` too small, or the genuinely relevant chunk simply isn't in the corpus.
- **Too much context** — retrieving so many chunks that the relevant fact is diluted or pushed past the model's effective attention (Chapter 8.35).
- **Stale documents** — the index contains an outdated version of a document that's since been superseded.
- **Duplicate documents** — the same content indexed multiple times, wasting top-k slots and skewing which sources appear to "agree."
- **Conflicting documents** — the corpus genuinely contains contradictory information (e.g. an old and new policy both present) and the model has no way to know which is authoritative without a signal from metadata (Chapter 8.24).
- **Hallucinated answers** — the model generates content not supported by the retrieved context, despite grounding instructions.
- **Missing citations** — the model produces an answer without attributing it to a source, or attributes it incorrectly.
- **Retrieval latency** — slow embedding or search calls that dominate end-to-end response time.
- **Indexing failures** — silent ingestion errors (Chapter 8.21) that leave documents partially or entirely unindexed.

31.2 Diagnostic Decision Tree

```
Answer is wrong or ungrounded
      ↓
Was the right chunk actually retrieved?
      │
      ├── No → Is it in the corpus at all?
      │           ├── No  → Ingestion gap — check Chapter 8.21/8.48
      │           └── Yes → Retrieval problem — check chunking (8.22),
      │                         embedding model fit (8.16), k, hybrid search (8.25)
      │
      └── Yes → Was it in the context sent to the LLM?
                  │
```

```
                    ├── No  → Context budgeting dropped it — check 8.35
                    └── Yes → Generation problem — check the prompt's
                              grounding instructions (8.10/8.34), or
                              this is a genuine hallucination despite
                              correct context
```

Chapter 32

Chapter 8.32 — RAG Evaluation

"The LLM gave a fluent answer" is not an evaluation metric. Fluency and correctness are independent — a RAG system can produce confident, well-written, completely wrong output. Evaluation has to check retrieval and generation separately, because a good answer with bad retrieval (the model got lucky from parametric knowledge) and a bad answer with good retrieval (the model failed to use good context) require different fixes.

32.1 Retrieval Metrics

- **Recall@k** — of the documents that are actually relevant to a query, what fraction were retrieved in the top-k?
- **Precision@k** — of the top-k retrieved documents, what fraction are actually relevant?
- **MRR (Mean Reciprocal Rank)** — the average of `1 / rank_of_first_relevant_result` across queries; rewards putting a relevant result near the top, not just somewhere in top-k.
- **Hit rate** — the fraction of queries for which at least one relevant document appears in the top-k.

32.2 Generation Metrics

- **Groundedness** — does every claim in the answer trace back to something actually present in the retrieved context? (Checkable via LLM-as-judge, human review, or claim-level entailment checking.)
- **Answer relevance** — does the answer actually address the question asked, independent of whether it's grounded?
- **Completeness** — does the answer cover everything the retrieved context supports, or does it omit relevant available information?
- **Citation correctness** — do the cited sources actually contain the claims attributed to them?

32.3 Building an Evaluation Dataset

At minimum, an evaluation set needs, per example:

```
{
  "question": "What is the maximum PTO carryover?",
  "expected_source_ids": ["doc_00931_chunk_012"],
  "expected_answer_summary": "5 days may be carried over into the next calendar year.",
  "category": "policy_lookup"
}
```

- **Best Practice**

Build your evaluation set from real or realistic user questions, not questions you write by paraphrasing the source document. Users ask questions in their own words, with typos, ambiguity, and phrasing that doesn't echo the document's vocabulary – an eval set built purely from document paraphrases will systematically overestimate retrieval quality.

Chapter 33

Chapter 8.33 — RAG Evaluation Project

Given a fixed set of documents, a fixed set of test questions with expected sources and expected answers, build a complete evaluation harness comparing at least two retrieval configurations.

```python
def evaluate_retrieval(eval_set, index, chunks, k=5):
    hits, reciprocal_ranks = 0, []
    for example in eval_set:
        results = retrieve(example["question"], index, chunks, k=k)
        retrieved_ids = [r["chunk_id"] for r in results]
        expected_ids = set(example["expected_source_ids"])

        hit = any(rid in expected_ids for rid in retrieved_ids)
        hits += int(hit)

        rank = next((i + 1 for i, rid in enumerate(retrieved_ids) if rid in expected_ids), None)
        reciprocal_ranks.append(1 / rank if rank else 0)

    return {
        "hit_rate": hits / len(eval_set),
        "mrr": sum(reciprocal_ranks) / len(eval_set),
    }

# Compare two configurations
config_a_metrics = evaluate_retrieval(eval_set, index_dense_only, chunks, k=5)
config_b_metrics = evaluate_retrieval(eval_set, index_hybrid, chunks, k=5)
print("Dense only:", config_a_metrics)
print("Hybrid:", config_b_metrics)
```

Report, per configuration: retrieval hit rate and MRR, answer quality (human-judged or LLM-judged groundedness/relevance on a sample), citation correctness rate, and end-to-end latency (Chapter 8.49).

Chapter 34

Chapter 8.34 — Hallucination Control

Hallucinations — fluent output not actually supported by the source of truth — cannot be fully eliminated in an LLM-based system. The goal is to reduce their frequency and, critically, to make the system's uncertainty visible rather than hidden.

34.1 Strategies

- **Retrieval grounding** — always base answers on retrieved context rather than the model's parametric memory alone (Chapter 8.27).
- **Explicit context instructions** — instruct the model directly to answer *only* from the provided context and to say so when it can't (Chapter 8.10's grounded prompt pattern).
- **Source citation** — requiring citations (Chapter 8.29) makes ungrounded claims easier to catch, both automatically and by the end user.
- **Confidence thresholds** — for retrieval, discard or flag results below a similarity-score threshold rather than always returning top-k regardless of how weak the match is.
- **Abstention** — explicitly instructing and testing that the model says "I don't know" or "the provided information doesn't cover this" when appropriate, rather than always attempting an answer.
- **Answerability detection** — a pre-generation check (rule-based or a small classifier/LLM call) that estimates whether the retrieved context actually contains an answer before generation runs, to route low-confidence cases differently (e.g. to a human, or to a broader search).
- **Retrieval validation** — a post-hoc check (e.g. an LLM-as-judge call) verifying that the final answer's claims are actually present in the retrieved context, used as a guardrail before the answer reaches the user.

```python
def answer_with_abstention_check(query, index, chunks):
    retrieved = retrieve(query, index, chunks, k=5)
    top_score = retrieved[0]["score"] if retrieved else 0.0

    if top_score < 0.35:  # threshold tuned against your own eval set
        return "I don't have enough relevant information to answer that confidently."

    prompt = build_prompt_with_citations(query, retrieved)
    return generate(prompt)
```

• Common Mistake

Marketing a RAG system internally as "hallucination-proof" because it's grounded in retrieved documents. Grounding substantially *reduces* hallucination risk relative to an ungrounded model, but the LLM can still misread, over-generalize from, or selectively ignore correct retrieved context. Evaluation (Chapter 8.32) and abstention (above), not a grounding claim alone, are what let you make an honest reliability statement about the system.

★ Quick Revision Sheet

Chapters 8.27–8.34, in one screen:

- RAG pipeline: load → clean → chunk → embed → index (ingestion time) → retrieve → rerank → context → LLM → answer (query time).
- Build raw-Python RAG before reaching for a framework – it makes every later abstraction legible instead of magic.
- Citation-aware RAG: number sources in the prompt, require the model to cite by number, then validate cited numbers against the actually-retrieved set in code – never trust a citation the model invents.
- Advanced patterns (query rewriting, HyDE, multi-query, contextual compression, parent-child, hierarchical retrieval) add real cost – reach for chunking and hybrid search fixes first.
- Diagnosing a bad answer: was the right chunk retrieved? was it in the final context sent to the LLM? only then is it a generation-layer problem.
- Evaluate retrieval (Recall@k, Precision@k, MRR, hit rate) and generation (groundedness, relevance, completeness, citation correctness) *separately* – fluency is not correctness.
- Hallucination cannot be eliminated, only reduced and made visible – grounding, citations, confidence thresholds, and abstention are the practical levers.

Chapter 35

Chapter 8.35 — Context Window Management

Every RAG call competes for a fixed token budget: system instructions, retrieved context, conversation history, and the model's own output all draw from the same context window.

35.1 Concepts

- **Token counting** — measuring the actual token cost of a prompt using the target model's tokenizer, not a word or character estimate (which can be off by 30%+ depending on language and content).
- **Context budgeting** — allocating the available token window across system prompt, retrieved context, conversation history, and reserved output space *before* assembling the final prompt.
- **Truncation** — cutting content (usually the least relevant retrieved chunk, or the oldest conversation turns) when the budget is exceeded.
- **Retrieval limits** — capping `k` not just for retrieval quality (Chapter 8.25) but as a hard context-budget constraint.
- **Summarization** — replacing older conversation history or long documents with a compressed summary to free up budget while retaining salient information.
- **Context compression** — more targeted than summarization: stripping irrelevant sentences from retrieved chunks specifically (related to Chapter 8.30's contextual compression concept), rather than compressing everything uniformly.
- **Priority ordering** — deciding what gets cut first when something must be cut (typically: lowest-relevance-score retrieved chunks before system instructions, and oldest conversation turns before recent ones).

```
import tiktoken  # or the target model's own tokenizer

def count_tokens(text: str, encoding_name: str = "cl100k_base") -> int:
    encoding = tiktoken.get_encoding(encoding_name)
    return len(encoding.encode(text))

def build_budgeted_context(retrieved_chunks: list[dict], max_context_tokens: int) -> str:
    # chunks are already sorted by relevance score, descending
    selected, used = [], 0
    for chunk in retrieved_chunks:
        chunk_tokens = count_tokens(chunk["text"])
        if used + chunk_tokens > max_context_tokens:
```

```
            break
        selected.append(chunk)
        used += chunk_tokens
    return "\n\n".join(c["text"] for c in selected)

def assemble_prompt(system_prompt, query, retrieved_chunks, total_window, reserved_output_tokens):
    budget_for_context = total_window - count_tokens(system_prompt) - count_tokens(query) - reserved_output_to
    context = build_budgeted_context(retrieved_chunks, budget_for_context)
    return system_prompt, context, query
```

- **Best Practice**

Always reserve output tokens *before* filling the context budget, not after. A prompt that consumes 99% of the context window leaves no room for the model to produce a complete answer, and many APIs will simply truncate the response mid-sentence rather than erroring.

Chapter 36

Chapter 8.36 — LLM Cost & Latency Optimization

36.1 Levers

- **Caching** — store and reuse results for repeated or near-duplicate queries (exact-match cache) or repeated prompt prefixes (prompt/context caching, supported natively by some providers) to avoid redundant LLM calls.
- **Batching** — grouping multiple requests into fewer API calls or fewer GPU forward passes where the workload allows (e.g. offline embedding of a large corpus).
- **Smaller models** — route simpler tasks (classification, extraction) to smaller/cheaper models and reserve larger models for tasks that need their reasoning capacity (model routing, below).
- **Shorter prompts** — trimming boilerplate system instructions and unnecessary verbosity directly reduces input token cost and latency.
- **Fewer retrieved chunks** — smaller `k` reduces both context tokens and (with reranking) the number of expensive reranker calls.
- **Embedding caching** — cache embeddings for documents that don't change between runs; never re-embed unchanged content.
- **Streaming** — doesn't reduce total cost, but substantially improves perceived latency for interactive use (Chapter 8.14).
- **Asynchronous requests** — issuing independent LLM/embedding calls concurrently rather than sequentially, when your workload has parallelizable calls (e.g. embedding multiple documents, or a multi-query retrieval pattern).
- **Model routing (concept)** — automatically directing a request to the cheapest model capable of handling it, based on task type or a lightweight classifier, rather than sending every request to the largest available model by default.

36.2 Measurement Framework

```
import time

def measure_call(fn, *args, **kwargs):
    start = time.perf_counter()
    result = fn(*args, **kwargs)
    elapsed = time.perf_counter() - start
    return result, elapsed
```

```python
# Track per-stage latency across a RAG call
timings = {}
retrieved, timings["retrieval"] = measure_call(retrieve, query, index, chunks)
reranked, timings["reranking"] = measure_call(rerank, query, retrieved)
response, timings["generation"] = measure_call(generate, build_prompt(query, reranked))
timings["total"] = sum(timings.values())
```

Track, per request: retrieval latency, reranking latency, LLM latency, total latency, input/output token counts, and estimated cost — Chapter 8.49 builds this into a full benchmarking harness.

Chapter 37

Chapter 8.37 — LLM Security Basics

37.1 Core Threats

- **Prompt injection** — a user directly instructs the model to ignore its system instructions or perform an unintended action.
- **Indirect prompt injection** — malicious instructions embedded in *content the model retrieves or reads*, not typed by the user directly. This is the RAG-specific version of the threat and is significantly harder to defend against, because the untrusted content arrives through the same channel as legitimate knowledge.
- **Malicious documents** — a document deliberately crafted (by whoever authored or can influence the corpus) to contain hidden instructions targeting the LLM rather than the human reader.
- **Data exfiltration risks** — an attacker crafting input that causes the model to leak system prompts, other users' data, or other documents' content it shouldn't disclose.
- **Sensitive data exposure** — retrieved context inadvertently containing information the current user isn't authorized to see (Chapter 8.38).
- **Unsafe tool instructions** — in agentic setups, injected content instructing the model to call tools (send an email, delete a file, make an API request) — largely out of scope for pure RAG but relevant the moment retrieval feeds an agent (Part 11).

37.2 RAG-Specific Injection Path

```
Untrusted Document
        ↓
Retrieved
        ↓
Presented as Context
        ↓
Potentially Influences Model
```

A document in your corpus does not have to be adversarially uploaded on purpose — it can simply be user-generated content (a support ticket, a wiki page anyone can edit) that happens to contain text engineered, deliberately or not, to look like an instruction.

37.3 Defensive Patterns

- **Delimiters** — clearly wrap retrieved content in explicit boundaries (Chapter 8.10) and instruct the model that content inside those boundaries is *data to reference*, never instructions to follow.
- **Least privilege** — the RAG system's downstream actions (if any — e.g. an agent that can act on retrieved content) should run with the minimum permissions needed, so even a successful injection has limited blast radius.
- **Output validation** — validate structured outputs (Chapter 8.12) and citations (Chapter 8.29) rather than trusting model output as if it always faithfully reflects your instructions.
- **Content provenance** — track and, where relevant, restrict which documents/sources are eligible for retrieval into higher-trust contexts.
- **Monitoring** — log retrieved content and generated output so unusual patterns (a document consistently correlating with off-policy responses) can be caught and investigated.

Advanced AI security — including systematic red-teaming and defenses for agentic tool-calling — is covered in depth in Part 12; this chapter is the baseline every RAG builder needs before that.

Chapter 38

Chapter 8.38 — RAG with Authorization

Retrieval-level access control: enforcing document permissions *at* the retrieval step, not after.

```
User
 ↓
Permission Check
 ↓
Allowed Documents
 ↓
Retrieval
 ↓
LLM
```

38.1 Why Filtering Only After Retrieval Is Dangerous

If retrieval searches the entire corpus and permission filtering happens only on the final output, two problems follow: first, a document the user isn't authorized to see can still occupy one of the limited top-k slots, silently degrading the quality of what the user *does* get back; second, and worse, if the filtering step has any bug or is accidentally skipped, unauthorized content reaches the LLM's context and potentially the user's screen. Filtering must happen *before or during* the similarity search, using the metadata access tags from Chapter 8.24, so unauthorized documents are never even candidates.

38.2 Practical Prototype

```python
def authorized_search(query: str, user, index, chunks: list[dict], k: int = 5):
    query_vec = embed_model.encode([query], normalize_embeddings=True).astype("float32")

    # Over-fetch, then filter by permission, since FAISS's flat index has
    # no native metadata filtering — a real vector DB (Chroma/Qdrant/Weaviate)
    # can push this filter down into the search itself.
    scores, indices = index.search(query_vec, k * 5)

    results = []
    for i, idx in enumerate(indices[0]):
        chunk = chunks[idx]
```

```
        if user_can_access(user, chunk):
            results.append({**chunk, "score": float(scores[0][i])})
        if len(results) >= k:
            break
    return results

def user_can_access(user, chunk: dict) -> bool:
    required_level = chunk.get("access_level", "public")
    return user.access_level >= ACCESS_LEVELS[required_level]
```

- **Common Mistake**

Implementing authorization as a filter on the *final generated answer* rather than on retrieval. By the time an answer is generated, unauthorized content has already been read by the model and may have influenced the response even if the literal source text isn't quoted back to the user. Enforce access control before content reaches the LLM's context, not after.

Chapter 39

Chapter 8.39 — LLM Observability Preview

A minimal set of signals every RAG system in production should log per request — full observability tooling (structured tracing, dashboards, alerting) is covered in depth in Part 12; this is the preview.

39.1 Signals to Capture

- **Latency** — end-to-end, and broken down by stage (retrieval, reranking, generation).
- **Token usage** — input and output tokens per request, for both embedding calls and LLM calls.
- **Retrieval latency** — isolated from generation latency, since these have different scaling behavior and different fixes.
- **Model latency** — the LLM call specifically.
- **Errors** — rate-limit hits, timeouts, validation failures, with enough context to reproduce.
- **Retrieval results** — which chunks were retrieved for which query, at minimum by ID, so retrieval quality issues are debuggable after the fact.
- **Prompt/version tracking** — which prompt template version and which model version produced a given response, so a quality regression can be traced to a specific change.

```
import logging
import time
import uuid

logger = logging.getLogger("rag_observability")

def observed_rag_call(query: str, index, chunks):
    request_id = str(uuid.uuid4())
    start = time.perf_counter()

    retrieved = retrieve(query, index, chunks, k=5)
    retrieval_latency = time.perf_counter() - start

    gen_start = time.perf_counter()
    response = generate(build_prompt(query, retrieved))
    generation_latency = time.perf_counter() - gen_start

    logger.info({
        "request_id": request_id,
```

```
        "query": query,
        "retrieved_chunk_ids": [c["chunk_id"] for c in retrieved],
        "retrieval_latency_ms": retrieval_latency * 1000,
        "generation_latency_ms": generation_latency * 1000,
        "total_latency_ms": (time.perf_counter() - start) * 1000,
    })
    return response
```

Chapter 40

Chapter 8.40 — RAG Deployment

A production-oriented RAG API, built with FastAPI.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import os
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("rag_api")

app = FastAPI(title="RAG API")

# Load index and clients once at startup, not per request
index, chunks = load_index_from_disk(os.environ["INDEX_PATH"])

class QueryRequest(BaseModel):
    question: str
    k: int = 5

class QueryResponse(BaseModel):
    answer: str
    sources: list[str]

@app.post("/query", response_model=QueryResponse)
def query(request: QueryRequest):
    if not request.question.strip():
        raise HTTPException(status_code=400, detail="question must not be empty")
    try:
        retrieved = retrieve(request.question, index, chunks, k=request.k)
        answer_text = generate(build_prompt_with_citations(request.question, retrieved))
        return QueryResponse(
            answer=answer_text,
            sources=[c["source"] for c in retrieved],
        )
    except Exception:
```

```
        logger.exception("query failed")
        raise HTTPException(status_code=500, detail="internal error processing query")

@app.get("/health")
def health():
    return {"status": "ok"}
```

```
Client
 ↓
FastAPI
 ↓
Retriever
 ↓
Vector Store
 ↓
LLM
 ↓
Response
```

40.1 Production Concerns

- **Environment variables** — `INDEX_PATH`, `ANTHROPIC_API_KEY`, and any config loaded from the environment, never hard-coded.
- **Configuration** — a settings object (e.g. `pydantic-settings`) rather than scattered `os.environ` calls throughout the codebase.
- **Logging** — structured logs at minimum for every request's latency and outcome (Chapter 8.39).
- **Error handling** — catch and log unexpected exceptions; return a generic error to the client rather than leaking internals (stack traces, file paths) in the response.
- **Request validation** — Pydantic models on every endpoint (as above) reject malformed requests before they reach business logic.
- **Streaming where appropriate** — a `/query/stream` endpoint using the pattern from Chapter 8.14 for interactive clients.

40.1.1 LAB 10 — Deploy RAG with FastAPI

Wrap the pipeline from Chapter 8.28/8.29 in a FastAPI app with `/query` and `/health` endpoints. Test with `curl` or an HTTP client. Confirm the API returns a structured error (not a stack trace) for an empty question.

Chapter 41

Chapter 8.41 — Dockerizing RAG

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV PYTHONUNBUFFERED=1

# Index and API key are provided at runtime, not baked into the image
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t rag-api:latest .

docker run -d \
  --name rag-api \
  -p 8000:8000 \
  -e ANTHROPIC_API_KEY="${ANTHROPIC_API_KEY}" \
  -e INDEX_PATH=/data/index.faiss \
  -v "$(pwd)/data:/data" \
  rag-api:latest
```

41.1 What to Cover

- **Dependency installation** — pin exact versions in `requirements.txt`; unpinned dependencies are a reproducibility hazard in a container image meant to be rebuilt later.
- **Environment variables** — passed at `docker run` time (`-e`) or via an env file, never baked into the image (a

baked-in API key ends up in the image layers, retrievable by anyone with the image).

- **Container execution** — `uvicorn` bound to `0.0.0.0` (not `127.0.0.1`) so it's reachable from outside the container.
- **Health check** — Docker's `HEALTHCHECK` lets orchestrators (Docker Compose, Kubernetes) detect and restart an unhealthy container automatically.
- **Volume considerations** — the FAISS index and any persistent data should live on a mounted volume, not inside the container's writable layer, so it survives container recreation and can be updated independently of the image.
- **Model/data storage** — for local-inference deployments, model weights are typically large enough to warrant their own volume or a dedicated model-serving layer rather than baking them into the application image.

41.1.1 LAB 11 — Dockerize the System

Containerize the FastAPI RAG app from Chapter 8.40. Build the image, run it with an environment variable for the API key and a mounted volume for the index file, and confirm `/health` responds correctly from outside the container.

★ Quick Revision Sheet

Chapters 8.35–8.41, in one screen:

- Context budgeting: reserve output tokens first, then fill remaining budget with the highest-relevance retrieved chunks; truncate lowest-relevance content first.
- Cost/latency levers: caching, batching, smaller/routed models, shorter prompts, fewer retrieved chunks, embedding caching, async calls, streaming (perceived latency only).
- Indirect prompt injection – malicious instructions arriving *through retrieved content* – is the RAG-specific security threat; defend with delimiters, least privilege, output validation, and monitoring.
- Enforce authorization at the retrieval step (before content reaches the LLM's context), never only on the final output.
- Minimum observability: per-request latency by stage, token usage, retrieved chunk IDs, errors, and prompt/-model version.
- Production RAG API = FastAPI + env-var config + structured logging + explicit error handling + Pydantic request validation.
- Docker: pin dependencies, pass secrets at runtime not build time, bind to `0.0.0.0`, use a `HEALTHCHECK`, mount index/data as volumes.

Chapter 42

Chapter 8.42 — LLM Project 1: Semantic Search

```
Documents
 ↓
Embeddings
 ↓
FAISS
 ↓
Query
 ↓
Top-k Results
```

Build a standalone semantic search engine over a document collection — no generation step, just retrieval. This is the smallest complete project in this Part and the right starting point if you're working through the labs sequentially.

Scope: ingest a document set → embed → build a FAISS `IndexFlatIP` index → accept a query → return ranked top-k results with scores and source metadata.

Build on: Chapters 8.15–8.20.

Evaluation: hand-labeled (query, expected document) pairs; report Recall@k and MRR (Chapter 8.32).

Benchmarking: measure embedding throughput (documents/sec at ingestion) and query latency (Chapter 8.49).

Chapter 43

Chapter 8.43 — LLM Project 2: Document Q&A

Build a complete single-corpus document question-answering system supporting PDF, TXT, and Markdown input.

Pipeline: ingestion (Chapter 8.21, format-aware per file type) → chunking (Chapter 8.22) → embedding (Chapter 8.15) → retrieval (Chapter 8.25) → LLM answer generation with citations (Chapter 8.29) → evaluation (Chapter 8.32).

```python
def ingest_document(path: str) -> list[dict]:
    ext = path.rsplit(".", 1)[-1].lower()
    if ext == "pdf":
        pages = extract_pdf_text(Path(path))
        raw_chunks = [recursive_split(p["text"]) for p in pages]
        return [
            {"text": c, "source": path, "page": p["page"]}
            for p, chunks in zip(pages, raw_chunks) for c in chunks
        ]
    elif ext in ("txt", "md"):
        text = Path(path).read_text(encoding="utf-8")
        return [{"text": c, "source": path, "page": None} for c in recursive_split(text)]
    else:
        raise ValueError(f"unsupported format: {ext}")
```

Acceptance criteria: given a held-out set of at least 15 questions with known-correct answers grounded in the corpus, the system must produce grounded, cited answers with a measured groundedness rate you report explicitly (not just "it seems to work").

Chapter 44

Chapter 8.44 — LLM Project 3: Multi-Document RAG

Extend Project 2 to support a corpus of many distinct documents with independent identity, tracked through the whole pipeline.

Additional requirements over Project 2:

- **Document IDs** — every chunk traces back to a stable `document_id`, independent of filename (filenames can collide or change).
- **Metadata** — full metadata schema per Chapter 8.24, including ingestion timestamp.
- **Filtering** — support querying a subset of documents (e.g. by `document_id` or a metadata tag) rather than always searching the full corpus.
- **Citations** — every answer must cite specific documents, not just "the corpus" in general.
- **Duplicate detection** — before indexing a new document, check whether its content (or near-duplicate content) is already indexed, to avoid the failure mode from Chapter 8.31.

```
import hashlib

def content_hash(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def is_duplicate(new_doc_text: str, existing_hashes: set[str]) -> bool:
    return content_hash(new_doc_text) in existing_hashes
```

Exact-hash duplicate detection catches identical re-uploads; near-duplicate detection (e.g. embedding the full document and checking similarity against existing document embeddings above a high threshold) is a reasonable stretch extension but not required for the base project.

Chapter 45

Chapter 8.45 — LLM Project 4: RAG API

Package Project 3 as a deployed, containerized API.

```
Frontend / Client
       ↓
FastAPI
       ↓
RAG Pipeline
       ↓
Vector Store
       ↓
LLM
```

Requirements:

- FastAPI service exposing `/query`, `/ingest` (or a separate offline ingestion script — either is acceptable, but the split must be a deliberate design decision, documented), and `/health` (Chapter 8.40).
- Environment-variable-based configuration; no hard-coded secrets.
- Structured logging and error handling (Chapter 8.39, 8.40).
- Request validation via Pydantic models.
- Dockerized (Chapter 8.41), with a working `docker build` and `docker run` that serves traffic on the container's exposed port.

Deliverable checklist: `Dockerfile`, `requirements.txt`, `main.py` (or equivalent), a `README.md` documenting how to build, run, and query the service, and an `.env.example` with no real secrets.

Chapter 46

Chapter 8.46 — LLM Project 5: Advanced RAG

A portfolio-grade system integrating the full breadth of this Part.

Required components:

- **Hybrid retrieval** — dense + sparse, combined via reciprocal rank fusion (Chapter 8.25).
- **Reranking** — cross-encoder reranking of the hybrid-retrieved candidate set (Chapter 8.26).
- **Metadata filtering** — at minimum, filtering by document type/source and by access level (Chapter 8.24, 8.38).
- **Citations** — full source-aware answer generation with validated citations (Chapter 8.29).
- **Evaluation** — a maintained evaluation set with retrieval and generation metrics (Chapter 8.32), re-run after any pipeline change.
- **Caching** — at minimum, embedding caching for unchanged documents (Chapter 8.36).
- **Observability hooks** — structured per-request logging of latency by stage and retrieved chunk IDs (Chapter 8.39).
- **Security checks** — retrieval-level authorization (Chapter 8.38) and delimiter-based defenses against indirect prompt injection (Chapter 8.37).

This project is the natural capstone for this Part — a system that would not be embarrassing to walk an interviewer through end to end, including its measured evaluation numbers and its known limitations.

Chapter 47

Chapter 8.47 — LLM Project Dataset & Environment Specifications

Every project in Chapters 8.42–8.46 must ship with the following, without exception.

47.1 Dataset Specification Template

```
Source: <exact dataset or document source>
URL: <official URL>
Format: <PDF / TXT / Markdown / CSV / JSON / mixed>
Sample count: <number of documents / rows, where applicable>
License: <license under which the dataset may be used>
Size: <total size on disk, or document/row count>
Preprocessing: <cleaning steps applied before ingestion>
Known issues: <e.g. scanned pages requiring OCR, inconsistent formatting>
```

47.2 Environment Specification Template

```
OS: <e.g. Ubuntu 22.04, macOS 14>
Python version: <e.g. 3.11>
Key library versions: <sentence-transformers, faiss-cpu, fastapi, anthropic, etc.>
CPU: <cores, architecture>
RAM: <minimum required>
GPU: <required / optional; model if required>
VRAM: <if GPU required>
Storage: <disk space for models + index + documents>
Expected latency: <p50/p95 per query, measured, not estimated>
Expected resource usage: <peak RAM/VRAM during embedding and generation>
```

47.3 Reproducibility Deliverables

Every project directory must contain:

```
requirements.txt       # pinned versions
environment_check.py   # verifies Python version, key packages, and (if applicable) GPU availability
README.md              # setup, run, and query instructions
.env.example            # documents required environment variables, with placeholder values only
```

```python
# environment_check.py — minimal example
import sys
import importlib

REQUIRED = ["fastapi", "faiss", "sentence_transformers", "anthropic", "pydantic"]

def check():
    assert sys.version_info >= (3, 10), "Python 3.10+ required"
    missing = []
    for pkg in REQUIRED:
        try:
            importlib.import_module(pkg)
        except ImportError:
            missing.append(pkg)
    if missing:
        print(f"Missing packages: {missing}")
        sys.exit(1)
    print("Environment OK.")

if __name__ == "__main__":
    check()
```

• Best Practice

Never commit a real API key to `.env.example` or anywhere in version control, including in commit history. Use placeholder values (`ANTHROPIC_API_KEY=your_key_here`) and document where to obtain a real one in the README.

★ Quick Revision Sheet

Chapters 8.42–8.47, in one screen:

- Project 1 (Semantic Search) → Project 2 (Document Q&A) → Project 3 (Multi-Document RAG) → Project 4 (RAG API) → Project 5 (Advanced RAG) is a deliberately incremental build order – each project's pipeline is a superset of the previous one's.
- Document identity (stable `document_id`, content-hash duplicate detection) becomes necessary the moment a corpus has more than one document worth distinguishing.
- Every project ships a dataset spec, an environment spec, and reproducibility files (`requirements.txt`, `environment_check.py`, `README.md`, `.env.example`) – this is not optional polish, it's the deliverable.
- Never commit real secrets to `.env.example` or version control history.

Chapter 48

Chapter 8.48 — LLM/RAG Debugging

Every row: **Symptom → Likely Cause → Diagnostic → Fix.**

Symptom	Likely Cause	Diagnostic	Fix
Model doesn't load	Insufficient VRAM/RAM for chosen `torch_dtype`	Check error for OOM; check `nvidia-smi` / system memory	Use a smaller model, quantize, or switch to `bfloat16`/`float16`
Tokenizer mismatch	Loaded tokenizer from a different model repo than the weights	Compare tokenizer's vocab size / special tokens to model config	Load tokenizer with `AutoTokenizer.from_pretrained(same_model_id)`
CUDA OOM mid-generation	Batch size or `max_new_tokens` too large for available VRAM	Reduce batch size incrementally; monitor `nvidia-smi` during generation	Lower batch size, enable gradient/activation-friendly settings, or use a smaller model/quantization
Slow inference	CPU-only inference, or unnecessarily large `max_new_tokens`	Confirm `device_map`/actual device used; time a single forward pass	Move to GPU if available; reduce `max_new_tokens`; batch requests
Poor retrieval	Chunking, embedding model mismatch, or `k` too low	Manually inspect top-k results for 5–10 known queries	Retune chunk size/overlap (8.22), try hybrid search (8.25), reconsider embedding model (8.16)
Irrelevant chunks retrieved	Embedding model not suited to domain/language	Compare similarity scores for relevant vs. irrelevant results	Evaluate a domain-specific or multilingual embedding model (8.16)
Hallucinations despite RAG	Weak grounding instructions, or context genuinely lacking the answer	Check whether the correct chunk was actually retrieved and in-budget	Strengthen grounded-prompt instructions (8.10), add abstention (8.34), improve retrieval
Missing citations	Prompt doesn't require citation format explicitly, or model ignores instruction	Inspect raw model output for citation markers	Use numbered-source citation prompt (8.29) and validate output format in code

Symptom	Likely Cause	Diagnostic	Fix
Malformed JSON output	No schema validation, or model wrapped output in prose/markdown fence	Log raw output before parsing	Add explicit "output only JSON" instruction; strip markdown fences before parsing; validate with Pydantic (8.12)
Rate-limit errors (`429`)	Exceeding provider's requests-per-minute or tokens-per-minute limit	Check response headers/error body for rate-limit details	Implement exponential backoff with jitter (8.13); reduce request concurrency
API timeout	No explicit timeout set, or network/provider latency spike	Check whether a timeout was configured at all	Set an explicit timeout; retry with backoff on timeout specifically
Vector dimension mismatch	Query embedded with a different model than the index was built with	Compare `index.d` (FAISS) to the query embedding's shape	Always embed queries and documents with the exact same model and version
Corrupted/unreadable index	Index file written incompletely (e.g. process killed mid-write) or read with wrong FAISS version	Attempt `faiss.read_index`; check file size/modification time against last known-good build	Rebuild the index from source documents; write to a temp path and atomically rename on success
Duplicate documents in results	No duplicate detection at ingestion	Check for repeated `document_id`/content hashes in the corpus	Add content-hash duplicate detection at ingestion (8.44)
Context overflow / truncated answer	Retrieved context + prompt exceeds context window, or output tokens weren't reserved	Count actual tokens sent vs. model's context limit	Apply context budgeting with reserved output tokens (8.35)

• Best Practice

When a bug report says "RAG gave a wrong answer" with no more detail, the first diagnostic step is always the same: log and inspect exactly which chunks were retrieved and exactly what final prompt was sent to the LLM for that query. The large majority of "the LLM is wrong" reports turn out to be "the retriever never found the right chunk" – a completely different fix.

Chapter 49

Chapter 8.49 — LLM/RAG Performance Benchmarking

49.1 What to Measure

- **Embedding speed** — documents (or characters/tokens) embedded per second, at your typical batch size.
- **Indexing speed** — vectors added to the index per second; matters for large-corpus ingestion jobs.
- **Retrieval latency** — time from query embedding to ranked candidate list.
- **Reranking latency** — time for the cross-encoder pass over the candidate shortlist.
- **LLM latency** — time-to-first-token and total generation time.
- **Total latency** — full end-to-end request time, which should roughly equal the sum of the above stages plus overhead.
- **Tokens/sec** — generation throughput, useful for comparing models/hardware configurations.
- **Memory** — peak RAM/VRAM during embedding and generation.
- **Cost** — per-request estimated cost (input tokens × input price + output tokens × output price, for API models; amortized compute cost for self-hosted).
- **Cache hit rate** — fraction of requests served from cache (Chapter 8.36), if caching is implemented.

49.2 Benchmark Script

```
import time
import statistics

def benchmark_pipeline(queries: list[str], index, chunks, runs_per_query: int = 3):
    stage_timings = {"retrieval": [], "reranking": [], "generation": [], "total": []}

    for query in queries:
        for _ in range(runs_per_query):
            t0 = time.perf_counter()

            t_r0 = time.perf_counter()
            retrieved = retrieve(query, index, chunks, k=20)
            stage_timings["retrieval"].append(time.perf_counter() - t_r0)
```

```
            t_rr0 = time.perf_counter()
            reranked = rerank(query, retrieved)[:5]
            stage_timings["reranking"].append(time.perf_counter() - t_rr0)

            t_g0 = time.perf_counter()
            _ = generate(build_prompt_with_citations(query, reranked))
            stage_timings["generation"].append(time.perf_counter() - t_g0)

            stage_timings["total"].append(time.perf_counter() - t0)

    report = {}
    for stage, values in stage_timings.items():
        report[stage] = {
            "p50_ms": statistics.median(values) * 1000,
            "p95_ms": (sorted(values)[int(len(values) * 0.95) - 1]) * 1000,
            "mean_ms": statistics.mean(values) * 1000,
        }
    return report
```

- **Best Practice**

Report p50 and p95 latency, not just the mean. A mean can look acceptable while a meaningful fraction of real requests time out or feel slow – p95 is what actually determines whether users notice a problem.

Chapter 50

Model Selection Decision Tree

```
Need an AI system
      ↓
What is required?
      │
      ├── Simple classification
      │       ↓
      │    Classical NLP
      │
      ├── Semantic similarity
      │       ↓
      │    Embeddings
      │
      ├── Knowledge-grounded answers
      │       ↓
      │    RAG
      │
      ├── Behavior/style/domain adaptation
      │       ↓
      │    Fine-tuning → Part 9
      │
      └── Autonomous multi-step behavior
              ↓
        Agents → Part 11
```

This is the single most important architectural decision in applied LLM engineering, and it is routinely made incorrectly by defaulting to whichever technique is currently most discussed. **Prompting, RAG, fine-tuning, and agents solve different problems** and are frequently combined, not chosen exclusively — a production agent (Part 11) commonly uses RAG internally as one of its tools, for instance.

50.1 Prompting vs. RAG vs. Fine-Tuning vs. Agents

Requirement	Prompting	RAG	Fine-Tuning
New knowledge	No (limited to what fits in context)	Yes	Only via retraining — expensive to keep current
Private documents	Only if pasted into the prompt each time	Yes, natively	Possible but couples private data into model weights
Behavior/style adaptation	Limited, via instructions	No — RAG changes what the model knows, not how it behaves	Yes — this is fine-tuning's core strength
Factual grounding	Weak — relies on parametric memory	Strong — answer is conditioned on retrieved source text	Weak on its own — factual grounding still needs retrieval or a very controlled training set
Cost	Lowest (no training, minimal engineering)	Moderate (retrieval infra + per-query LLM cost)	Highest (training compute + data curation)
Data requirements	None	A document corpus	A curated, often large, labeled training dataset
Updating knowledge	Instant (edit the prompt)	Fast (re-index changed documents)	Slow (requires retraining or an adapter update)
Deployment complexity	Lowest	Moderate (vector DB, ingestion pipeline)	Highest (training pipeline, evaluation, deployment of new weights)

Fine-tuning is not a replacement for retrieval when the primary requirement is access to changing or private knowledge. Fine-tuning bakes patterns into weights at a fixed point in time; it does not give a model an efficient way to look up a specific fact from a large, frequently-updated document set. Part 9 covers fine-tuning's actual strengths (behavior, style, format-following, domain-specific reasoning patterns) in depth — those strengths are different from, and complementary to, what RAG provides.

50.2 RAG vs. Long Context

Larger context windows have made "just put everything in the prompt" a real alternative to retrieval for some workloads — but the tradeoffs are real, not hypothetical:

- **Context cost** — every token in a long context is paid for (in latency and, for API models, in money) on *every single request*, even the requests where most of that context turns out to be irrelevant. Retrieval pays a smaller, targeted cost per request.
- **Latency** — processing a very long context, even from a model that technically supports it, adds real latency; a focused retrieved context is faster to process.
- **Relevance / noisy context** — models do not attend uniformly well across very long contexts in practice (retrieval of the specific relevant fact can degrade as irrelevant surrounding text grows), so "fits in the context window" is not the same as "the model will reliably use it correctly."

Scenario-based recommendations:

- Small, static, frequently-fully-relevant document set (e.g. a single product manual under a few thousand tokens) → long context alone may be sufficient, with lower engineering overhead than building a retrieval pipeline.
- Large, growing, or frequently-updated corpus → retrieval is necessary regardless of context window size, purely on cost and relevance-precision grounds.

- Corpus where most content is irrelevant to any given query → retrieval, to avoid diluting the model's attention with noise even if the raw token count would technically fit.
- High query volume in production → retrieval, because the per-request cost of processing a large fixed context on every call compounds quickly at scale.

• Interview Trap

"Context windows are huge now, so RAG is becoming obsolete." This is a common but incomplete claim: long context reduces *some* reasons to use RAG (small static corpora), but doesn't address per-request cost at scale, doesn't reliably eliminate noisy-context relevance degradation, and doesn't help with corpora larger than any context window (most enterprise document sets). The two are complementary tools with different cost/relevance profiles, not a linear replacement relationship.

Chapter 51

Chapter 8.50 — LLM Engineering Interview Preparation

120 questions spanning conceptual, coding, debugging, architecture, system-design, and scenario-based formats. Difficulty tags: EASY / MEDIUM / HARD.

51.1 LLMs & Tokenization

IQ1. What distinguishes a base model from an instruction-tuned model? **DIFFICULTY: EASY** *Companies: Google, Amazon* A base model is trained only on next-token prediction over raw text and continues a prompt rather than treating it as a task. An instruction-tuned model is further trained on (instruction, response) pairs so it responds to the prompt as a request to fulfill.

IQ2. Why can't you use one model's tokenizer with a different model's weights? **DIFFICULTY: EASY** *Companies: Meta, Microsoft* Token IDs only have meaning relative to the embedding table they index into. A different tokenizer's IDs point to arbitrary, unrelated positions in the model's embedding table, producing effectively random input.

IQ3. What is a context window, and what happens if you exceed it? **DIFFICULTY: EASY** *Companies: OpenAI, Anthropic* It's the maximum number of tokens (input plus, for most APIs, output) a model can process in one call. Exceeding it causes an error, silent truncation, or — depending on the provider — the request being rejected outright.

IQ4. Why do subword tokenizers split words like "unbelievable" into multiple pieces? **DIFFICULTY: MEDIUM** *Companies: Google* Subword tokenization keeps the vocabulary size manageable while still handling rare or unseen words by composing them from known, more frequent pieces, avoiding the combinatorial explosion of a whole-word vocabulary.

IQ5. What is the practical difference between a decoder-only and an encoder-decoder architecture? **DIFFICULTY: MEDIUM** *Companies: Amazon* A decoder-only model generates autoregressively with causal (left-to-right) attention and is the dominant architecture for chat LLMs. An encoder-decoder model reads the full input bidirectionally in an encoder, then generates output conditioned on that representation — common in translation and some structured-generation tasks.

IQ6. Why is "the weights are downloadable" not the same as "this model is open source"? **DIFFICULTY: MEDIUM** *Companies: Meta* Open source, in the traditional sense, requires the training code and training data (or a reproducible description) to also be freely available under an appropriate license. Most downloadable LLMs are open-weight only — the weights are public, but the training data and pipeline are not, and licenses often carry usage restrictions.

51.2 Inference & Generation Parameters

IQ7. What does `temperature=0` do, and is it equivalent to greedy decoding? **DIFFICULTY: EASY** *Companies: OpenAI* It sharpens the probability distribution maximally before sampling, and in most implementations is treated as equivalent to always picking the highest-probability token — i.e., greedy decoding.

IQ8. Explain the difference between top-k and top-p sampling. **DIFFICULTY: MEDIUM** *Companies: Anthropic, Google* Top-k restricts sampling to a fixed number of the highest-probability tokens regardless of how confident the model is. Top-p restricts sampling to the smallest set of tokens whose cumulative probability exceeds a threshold, so the candidate pool size adapts to the model's confidence at each step.

IQ9. Why does high temperature increase both creativity and incoherence risk? **DIFFICULTY: MEDIUM** *Companies: Meta* Increasing temperature flattens the probability distribution, giving lower-probability (often less coherent) tokens a meaningfully higher chance of being sampled alongside more diverse, creative choices — the two effects come from the same mechanism.

IQ10. Does lowering temperature reduce hallucination? **DIFFICULTY: HARD** *Companies: Anthropic* Not reliably. Lower temperature makes output more deterministic, but a deterministic, confidently-generated answer can still be a fully deterministic wrong answer. Grounding and retrieval quality control hallucination; decoding strategy controls randomness.

IQ11. What is repetition penalty, and when would you increase it? **DIFFICULTY: EASY** *Companies: Amazon* It down-weights tokens that already appeared in the generated sequence. Increase it when you observe looping or verbatim repetition in longer generations.

IQ12. Why is beam search rarely used for modern open-ended chat generation? **DIFFICULTY: HARD** *Companies: Google* Beam search optimizes for the highest-scoring overall sequence, which tends to produce generic, repetitive, "safe" text over long open-ended generation — it works better for short, well-defined outputs like translation than for creative or conversational text.

51.3 Chat Templates & Prompting

IQ13. Why can manually concatenating "System: … User: …" break a chat model? **DIFFICULTY: MEDIUM** *Companies: Meta, Mistral* Chat models are trained with specific special tokens marking role boundaries, defined per model family in a chat template. Hand-written text that doesn't match that exact structure is out-of-distribution input, which can cause the model to ignore instructions or fail to stop generating correctly.

IQ14. What does `apply_chat_template` do, and why should you always use it? **DIFFICULTY: EASY** *Companies: Hugging Face* It formats a list of role-tagged messages into the exact prompt string (including special tokens) the specific model was trained on, read from the model's own tokenizer config — guaranteeing the format matches training, unlike hand-written formatting.

IQ15. Give an example of a negative constraint in a prompt and explain why it's needed. **DIFFICULTY: EASY** *Companies: Startup, general* "Do not include a preamble before the answer." It's needed when a model's default verbose/conversational behavior conflicts with a specific output requirement, such as an API response that must start immediately with structured data.

IQ16. Why should retrieved or user-supplied content be wrapped in explicit delimiters in a prompt? **DIFFICULTY: MEDIUM** *Companies: Anthropic* Delimiters (triple quotes, XML tags) make the boundary between instructions and data explicit, reducing the chance the model misreads content as an instruction — and this is a first line of defense against prompt injection.

51.4 Structured Outputs

IQ17. Why is `json.loads()` on raw model output, with no validation, a production risk? **DIFFICULTY: MEDIUM** *Companies: Amazon* Models can wrap JSON in markdown fences, add trailing prose, omit required fields, or emit subtly wrong types. Unvalidated parsing either crashes unpredictably or silently passes malformed data downstream.

IQ18. What is constrained decoding, and how does it differ from "asking nicely" for JSON in the prompt? **DIFFICULTY: HARD** *Companies: OpenAI* Constrained decoding masks the logits at each generation step so only tokens consistent with a grammar/schema can ever be sampled, making invalid structure impossible rather than merely discouraged. Prompted JSON has no such guarantee — the model can still deviate.

IQ19. Why validate structured LLM output against a schema even when using constrained decoding? **DIFFICULTY: MEDIUM** *Companies: Google* Constrained decoding guarantees syntactic validity (well-formed JSON matching the grammar) but not semantic correctness (a field could still contain a nonsensical or wrong value within its type constraint) — schema validation plus downstream logic checks are still needed.

51.5 API Engineering

IQ20. Which HTTP error classes should trigger a retry, and which shouldn't? **DIFFICULTY: EASY** *Companies: Amazon, Anthropic* Retry on `5xx` server errors, `429` rate limits, and timeouts (all transient). Don't blindly retry `4xx` client errors like `400` (malformed request) or `401` (bad credentials) — the request will fail identically every time without a code change.

IQ21. Why use exponential backoff with jitter instead of fixed-interval retries? **DIFFICULTY: MEDIUM** *Companies: Google* Fixed-interval retries from many clients synchronize and can cause repeated simultaneous request bursts ("thundering herd"). Exponential backoff spaces retries out over time, and jitter (randomness) desynchronizes clients from each other.

IQ22. Where should an API key live in a production system? **DIFFICULTY: EASY** *Companies: general* In environment variables or a secrets manager, injected at runtime — never hard-coded in source or committed to version control.

51.6 Streaming

IQ23. What problem does streaming solve, and what problem does it not solve? **DIFFICULTY: EASY** *Companies: OpenAI* It reduces perceived latency by showing partial output as it's generated. It does not reduce total generation time, total token cost, or the time until the full response is complete.

IQ24. What must client code handle robustly when consuming a streamed LLM response? **DIFFICULTY: MEDIUM** *Companies: Anthropic* Mid-stream connection drops, and chunk boundaries that don't align cleanly with word or token boundaries — the client must be able to reassemble partial output correctly regardless of how it was chunked.

51.7 Embeddings

IQ25. How does an embedding model's training objective differ from a generative LLM's? **DIFFICULTY: MEDIUM** *Companies: Google, Meta* Embedding models are typically trained with a contrastive objective that pulls semantically similar texts' vectors together and pushes dissimilar ones apart. Generative LLMs are trained on next-token prediction, which optimizes for fluent continuation, not for geometric similarity structure.

IQ26. Can you use a chat LLM's hidden states directly as embeddings? **DIFFICULTY: HARD** *Companies: Anthropic* Not reliably without specific adaptation — nothing in next-token-prediction training guarantees that semantically sim-

ilar texts land close together in that hidden-state space, which is exactly the property a dedicated embedding model is trained to have.

IQ27. Why normalize embeddings before indexing? **DIFFICULTY: MEDIUM** *Companies: Amazon* On L2-normalized vectors, inner product ranking becomes equivalent to cosine similarity ranking, and inner product is cheaper to compute — normalizing once at index time avoids recomputing normalization per query and lets you use faster similarity search implementations.

IQ28. Why might a general-purpose embedding model perform poorly on a legal or medical corpus? **DIFFICULTY: MEDIUM** *Companies: Startup, healthcare* General-purpose models are trained predominantly on broad web/-general text and may not capture the specific semantic distinctions and terminology relationships that matter within a specialized domain's vocabulary.

IQ29. What's the tradeoff between a small and a large embedding model? **DIFFICULTY: EASY** *Companies: general* Smaller models are faster and cheaper to run at scale but may have a lower retrieval-quality ceiling; larger models often retrieve more accurately but cost more in compute, memory, and latency.

51.8 Vector Search & FAISS

IQ30. What is the difference between `IndexFlatL2` and `IndexFlatIP` in FAISS? **DIFFICULTY: EASY** *Companies: Meta* `IndexFlatL2` ranks by Euclidean distance (smaller = more similar); `IndexFlatIP` ranks by inner product (larger = more similar). On normalized vectors, `IndexFlatIP` ranking is equivalent to cosine similarity ranking.

IQ31. Why is a flat FAISS index described as "exact," and what's the tradeoff? **DIFFICULTY: MEDIUM** *Companies: Amazon* It computes similarity against every stored vector with no approximation, guaranteeing correct top-k results, but at linear cost in the number of vectors — it doesn't scale as well as approximate indexes (e.g. IVF, HNSW) at very large corpus sizes.

IQ32. What does normalizing vectors before adding them to `IndexFlatIP` actually accomplish? **DIFFICULTY: MEDIUM** *Companies: Google* It removes the influence of vector magnitude from the inner-product score, leaving only orientation — which is exactly what cosine similarity measures — so inner product search on normalized vectors reproduces cosine similarity rankings at lower compute cost.

IQ33. Why is FAISS described as a library rather than a database? **DIFFICULTY: EASY** *Companies: general* It provides fast similarity search primitives but no built-in persistence layer, metadata storage, filtering, or server/API — those have to be built around it, unlike Chroma, Qdrant, Weaviate, or Milvus, which provide them out of the box.

51.9 Vector Databases

IQ34. When would you choose Qdrant or Weaviate over raw FAISS? **DIFFICULTY: MEDIUM** *Companies: Startup* When you need built-in metadata filtering, persistence, and often a managed cloud deployment option without building that infrastructure yourself — the tradeoff is added operational complexity and dependency versus FAISS's minimal footprint.

IQ35. What does "filtering pushed down into the search" mean, and why does it matter? **DIFFICULTY: HARD** *Companies: Amazon* It means the metadata filter (e.g. access level, document type) is applied *during* the similarity search itself rather than after retrieving an unfiltered top-k and filtering client-side — this avoids wasting top-k slots on results that will be discarded and, for authorization use cases, avoids ever surfacing unauthorized content as a candidate.

IQ36. What is metadata, in the vector database sense, and give three concrete uses. **DIFFICULTY: EASY** *Companies: general* Structured key/value data attached to each vector (source, page, access level, timestamp). Uses: citation display, retrieval-time authorization filtering, and debugging retrieval quality by tracing a result back to its source.

51.10 Chunking

IQ37. Why is chunk size described as one of the highest-leverage decisions in a RAG pipeline? **DIFFICULTY: HARD** *Companies: Anthropic, OpenAI* Chunk size directly determines whether a specific fact ends up isolated and retrievable or diluted/split across chunk boundaries. Even a strong embedding model and a strong LLM can't compensate for a chunk that either cuts a fact in half or buries it in irrelevant surrounding text.

IQ38. What is recursive splitting, and why is it preferred over fixed-size chunking? **DIFFICULTY: MEDIUM** *Companies: Google* It tries to split on the largest natural structural boundary first (section, then paragraph, then sentence), falling back to smaller boundaries only when a piece still exceeds the size limit — this respects document structure while still guaranteeing a size ceiling, unlike fixed-size chunking, which cuts blindly at a character count.

IQ39. Why use overlap between chunks? **DIFFICULTY: EASY** *Companies: Amazon* So information near a chunk boundary isn't orphaned from its surrounding context in either the preceding or following chunk — a fact split across a boundary has a better chance of being interpretable in at least one of the two overlapping chunks.

IQ40. What is semantic chunking, and why isn't it always the best choice? **DIFFICULTY: HARD** *Companies: Startup* Splitting based on detected topic shifts (e.g. via sentence-embedding similarity drops) rather than a fixed size or structural boundary. It's more adaptive but more expensive to compute, and its benefit over simpler recursive splitting is workload-dependent — it should be validated against an evaluation set, not assumed superior.

51.11 Metadata & Retrieval

IQ41. Why does every chunk need a `document_id` separate from its filename? **DIFFICULTY: MEDIUM** *Companies: Amazon* Filenames can collide, change, or be reused, while a stable, independent document identifier lets you track, filter, deduplicate, and update documents reliably regardless of filename churn.

IQ42. Compare BM25 and dense embedding retrieval. **DIFFICULTY: MEDIUM** *Companies: Google, Meta* BM25 (sparse/keyword) excels at exact-match precision on rare terms, IDs, and proper nouns but misses paraphrases entirely. Dense embedding retrieval captures semantic/paraphrase similarity well but can underperform on exact-match cases the embedding model saw rarely during training.

IQ43. What is reciprocal rank fusion, and why is it used to combine retrieval signals? **DIFFICULTY: HARD** *Companies: Anthropic* A method for merging two differently-scaled ranked lists by scoring each item as the sum of `1/(rank+1)` across the lists it appears in, then re-ranking by that combined score. It avoids needing to calibrate BM25 and embedding-similarity scores against each other directly, since it only relies on rank position.

IQ44. When is metadata filtering applied before vs. after similarity search, and why does it matter? **DIFFICULTY: HARD** *Companies: Amazon* Ideally before/during search (pushed down), so the search operates only over the allowed candidate set. Filtering only after search risks unauthorized or irrelevant documents occupying limited top-k slots, or — for access control specifically — being read by the LLM before being filtered out.

51.12 Reranking

IQ45. What is the structural difference between a bi-encoder and a cross-encoder? **DIFFICULTY: MEDIUM** *Companies: Google, Meta* A bi-encoder embeds the query and each document independently and compares vectors afterward, making it pre-computable and indexable. A cross-encoder takes the query and document together as joint input and outputs one relevance score, capturing interaction directly but requiring a fresh forward pass per pair at query time.

IQ46. Why retrieve a wide candidate pool (e.g. top 20) before reranking down to a small final set (e.g. top 5)? **DIFFICULTY: MEDIUM** *Companies: Amazon* The cheaper bi-encoder retriever can rank a genuinely relevant document too low to make a small top-k directly, because it never lets the query and document interact. A wider pool increases the chance the truly relevant documents are present for the more accurate cross-encoder to correctly promote.

IQ47. Why can't a cross-encoder be used as the primary retriever over an entire large corpus? **DIFFICULTY: MEDIUM** *Companies: Startup* Because it can't be pre-indexed — every query-document pair requires its own forward pass at query time, which is computationally infeasible over an entire large corpus but tractable over a small shortlist.

51.13 RAG Fundamentals

IQ48. Walk through the full RAG pipeline stage by stage. **DIFFICULTY: EASY** *Companies: general* Load → clean → chunk → embed → index (ingestion time), then retrieve → rerank → assemble context → LLM generation → answer (query time).

IQ49. Why is it useful to distinguish ingestion-time stages from query-time stages? **DIFFICULTY: MEDIUM** *Companies: Amazon* They have entirely different latency budgets and failure modes — ingestion can tolerate minutes per document and batch processing, while query-time stages need to complete within an interactive response-time budget; conflating them leads to confused system design and unrealistic performance expectations.

IQ50. Why build a RAG pipeline in raw Python before using a framework? **DIFFICULTY: MEDIUM** *Companies: Anthropic* Frameworks abstract over exactly the mechanics (chunking, embedding, retrieval, prompt assembly) covered in this Part. Understanding the raw pipeline first makes those abstractions legible and debuggable instead of opaque, and makes it possible to diagnose failures that occur inside the framework's black box.

51.14 Citations

IQ51. Why should citation numbers be validated in code rather than trusted from model output? **DIFFICULTY: MEDIUM** *Companies: Google* Because a model can cite a number that doesn't correspond to any actually-retrieved source, or misattribute a claim to the wrong source — validating cited numbers against the real retrieved set catches these failures before they reach the user.

IQ52. What's the advantage of numbered-source citation over asking the model to reproduce filenames from memory? **DIFFICULTY: MEDIUM** *Companies: Amazon* Requiring citation by number (mapped explicitly in the prompt) is far more reliable than trusting the model to accurately recall and reproduce exact filenames or page numbers, which it can get subtly wrong even when otherwise well-grounded.

51.15 Advanced RAG Patterns

IQ53. What is HyDE, and what's the intuition behind it? **DIFFICULTY: HARD** *Companies: Anthropic, Google* Hypothetical Document Embeddings: generate a hypothetical answer to the query with an LLM first, then embed and search with that hypothetical answer instead of the raw query, on the theory that an answer's embedding is geometrically closer to relevant documents than a short question's embedding is.

IQ54. What is parent-child retrieval, and what problem does it solve? **DIFFICULTY: HARD** *Companies: Meta* Indexing small, precise chunks for accurate retrieval matching, but returning the larger "parent" section they belong to as the actual context sent to the LLM — combining retrieval precision with generation-time completeness.

IQ55. Why shouldn't advanced RAG patterns be your first fix for poor retrieval quality? **DIFFICULTY: MEDIUM** *Companies: Startup* In practice, chunking strategy and basic hybrid search fix the majority of retrieval-quality problems at much lower added cost/latency than techniques like HyDE or multi-query retrieval, which add real overhead. Validate that a simpler fix isn't sufficient before reaching for more complex patterns.

51.16 RAG Failure Modes & Debugging

IQ56. A user reports a wrong RAG answer. What's your first diagnostic step? **DIFFICULTY: MEDIUM** *Companies: Amazon, Anthropic* Log and inspect exactly which chunks were retrieved and exactly what final prompt was sent to the LLM for that specific query — this immediately distinguishes a retrieval problem from a generation problem.

IQ57. What's the difference between "insufficient retrieval" and "irrelevant retrieval" as failure modes? **DIFFICULTY: MEDIUM** *Companies: Google* Insufficient retrieval means k is too small or the relevant chunk simply isn't in the corpus at all. Irrelevant retrieval means the retriever returns topically-similar but non-answering chunks — a ranking/matching problem, not a coverage problem.

IQ58. How can conflicting documents in a corpus cause a RAG failure that isn't the model's fault? **DIFFICULTY: HARD** *Companies: Startup* If the corpus genuinely contains contradictory information (e.g. an old and updated policy both present), the model has no way to know which is authoritative unless metadata (e.g. a timestamp or a superseded flag) signals it — the failure is a corpus/metadata gap, not a model flaw.

51.17 Evaluation

IQ59. Why is "the LLM gave a fluent answer" not a valid evaluation metric? **DIFFICULTY: EASY** *Companies: Anthropic, OpenAI* Fluency and correctness are independent — a system can produce confident, well-written, completely wrong output. Evaluation has to measure groundedness and relevance explicitly, not just readability.

IQ60. Define Recall@k and Precision@k in the retrieval context. **DIFFICULTY: EASY** *Companies: Google* Recall@k: of all documents actually relevant to a query, what fraction appear in the top-k retrieved. Precision@k: of the top-k retrieved documents, what fraction are actually relevant.

IQ61. What does MRR capture that hit rate doesn't? **DIFFICULTY: MEDIUM** *Companies: Amazon* MRR rewards ranking a relevant result *near the top*, not just having it appear anywhere within top-k — hit rate treats a relevant result at rank 1 and rank k identically, while MRR does not.

IQ62. Why should retrieval and generation be evaluated separately? **DIFFICULTY: MEDIUM** *Companies: Anthropic* A good answer can result from bad retrieval plus lucky parametric knowledge, and a bad answer can result from good retrieval plus a generation failure — these require entirely different fixes, so conflating them into one end-to-end "did it work" metric hides the actual problem location.

IQ63. Why should an evaluation set be built from real or realistic user questions rather than document paraphrases? **DIFFICULTY: HARD** *Companies: Startup* Users phrase questions in their own words, with ambiguity and vocabulary that doesn't echo the source document — an eval set built by paraphrasing the document systematically overestimates retrieval quality relative to real usage.

51.18 Hallucination

IQ64. Can hallucination be fully eliminated in an LLM-based system? **DIFFICULTY: EASY** *Companies: general* No — it can be substantially reduced through grounding, citations, and abstention, but not fully eliminated; an honest system design acknowledges and measures residual hallucination rate rather than claiming immunity.

IQ65. What is abstention, and why is testing for it important? **DIFFICULTY: MEDIUM** *Companies: Anthropic* Explicitly instructing (and verifying) that the model says "I don't know" when the retrieved context doesn't support an answer, rather than always attempting one — without testing this behavior explicitly, you don't actually know whether the model will abstain when it should.

IQ66. Why is "we ground answers in retrieved documents" not the same as "hallucination-proof"? **DIFFICULTY: MEDIUM** *Companies: Google* Grounding reduces hallucination risk relative to an ungrounded model, but the LLM can

still misread, over-generalize from, or selectively ignore correct retrieved context — grounding is a strong mitigation, not a guarantee.

51.19 Context Window Management

IQ67. Why should output tokens be reserved before filling the context budget with retrieved content? **DIFFICULTY: MEDIUM** *Companies: Amazon* If retrieved context consumes nearly the entire window, the model has too little room left to produce a complete answer, and many APIs will silently truncate the response rather than error — reserving output space up front avoids this.

IQ68. What's the difference between truncation and summarization as context-management strategies? **DIFFICULTY: EASY** *Companies: general* Truncation simply drops the lowest-priority content (e.g. oldest conversation turns, lowest-relevance chunks) when the budget is exceeded. Summarization compresses that content into a shorter form that retains salient information rather than discarding it outright.

IQ69. Why is token counting more reliable than word or character counting for budgeting? **DIFFICULTY: MEDIUM** *Companies: Google* Token-to-word ratio varies substantially by language and content type (code, non-English text especially), so word/character estimates can be off by a large margin relative to the model's actual tokenizer output.

51.20 Security

IQ70. What is indirect prompt injection, and why is it specific to RAG? **DIFFICULTY: HARD** *Companies: Anthropic, Google* Malicious instructions embedded in content the model *retrieves and reads* rather than typed directly by the user — it's RAG-specific because untrusted content arrives through the same channel (retrieved context) as legitimate knowledge, making it harder to distinguish from trusted instructions.

IQ71. Give two concrete defenses against indirect prompt injection in RAG. **DIFFICULTY: MEDIUM** *Companies: Amazon* Wrapping retrieved content in explicit delimiters with instructions that content inside is data, not instructions to follow; and validating/constraining any downstream actions the model can take so a successful injection has limited blast radius (least privilege).

IQ72. Why does a document not need to be adversarially crafted on purpose to cause an injection risk? **DIFFICULTY: HARD** *Companies: Startup* User-generated content in a corpus (support tickets, editable wiki pages) can incidentally contain text that reads like an instruction to a model, even without malicious intent — the risk exists any time untrusted or loosely-governed content is retrievable.

51.21 RAG with Authorization

IQ73. Why is filtering by permission only on the final generated answer considered dangerous? **DIFFICULTY: HARD** *Companies: Amazon, Google* Unauthorized content can already occupy limited top-k slots (degrading what the authorized user receives) or, more seriously, has already been read by the model and may have influenced the response before any output-stage filter runs — and a bug or skipped step in output filtering directly leaks unauthorized content.

IQ74. Where should access-control checks be enforced in a RAG pipeline? **DIFFICULTY: MEDIUM** *Companies: Amazon* At or before the retrieval step, using metadata access tags, so unauthorized documents are never candidates for retrieval in the first place — not as a post-hoc filter on the final answer.

51.22 Observability

IQ75. Name four signals a minimal RAG observability setup should capture per request. **DIFFICULTY: EASY** *Companies: general* Any four of: end-to-end and per-stage latency, token usage, retrieved chunk IDs, errors, and promp-

t/model version.

IQ76. Why log retrieved chunk IDs, not just the final answer text? **DIFFICULTY: MEDIUM** *Companies: Anthropic* It lets you debug retrieval-quality issues after the fact — tracing a wrong or surprising answer back to exactly which chunks were retrieved, rather than only seeing the opaque final output.

51.23 Deployment & Cost/Latency

IQ77. Why is caching described as the single highest-leverage cost/latency optimization? **DIFFICULTY: MEDIUM** *Companies: Google* It avoids redundant, full-cost LLM/embedding calls entirely for repeated or near-duplicate work, whereas most other optimizations (smaller prompts, fewer chunks) reduce the cost of each call rather than eliminating calls outright.

IQ78. What is model routing, and when is it useful? **DIFFICULTY: MEDIUM** *Companies: Amazon* Automatically directing a request to the cheapest model capable of handling it (e.g. based on task type or a lightweight classifier) rather than sending every request to the largest available model — useful when a workload has a mix of simple and complex requests.

IQ79. Why report p95 latency in addition to mean latency? **DIFFICULTY: MEDIUM** *Companies: Anthropic* A mean can look acceptable even while a meaningful fraction of real requests are slow or time out — p95 reflects what a non-trivial share of users actually experience, which the mean can hide.

IQ80. Why does streaming not reduce total cost? **DIFFICULTY: EASY** *Companies: general* Streaming changes when tokens are delivered to the client, not how many tokens are generated or their price — total token count and total generation time are unchanged.

51.24 FastAPI / Docker Deployment

IQ81. Why load the vector index once at application startup rather than per request? **DIFFICULTY: EASY** *Companies: Amazon* Loading a potentially large index from disk on every request adds significant, unnecessary latency and I/O load; loading once at startup and keeping it in memory serves all subsequent requests efficiently.

IQ82. Why should Docker containers bind to `0.0.0.0` rather than `127.0.0.1`? **DIFFICULTY: MEDIUM** *Companies: general* Binding to `127.0.0.1` (localhost) inside the container only accepts connections from within the container itself, making the service unreachable from outside — `0.0.0.0` accepts connections on any network interface, which is required for host/port-mapped traffic to reach it.

IQ83. Why should secrets be passed at `docker run` time rather than baked into the image at build time? **DIFFICULTY: MEDIUM** *Companies: Google, Amazon* A secret baked into an image layer during build remains retrievable by anyone with access to the image, even if later layers appear to remove it. Passing secrets at runtime (via `-e` or a mounted secrets file) keeps them out of the distributable image entirely.

IQ84. What does a Docker `HEALTHCHECK` enable that a container without one lacks? **DIFFICULTY: EASY** *Companies: general* It lets an orchestrator (Docker Compose, Kubernetes) automatically detect an unhealthy container and restart or replace it, rather than continuing to route traffic to a silently broken instance.

51.25 Debugging Scenarios

IQ85. A FAISS index loads without error but every query returns nonsensical results. What's the likely cause? **DIFFICULTY: HARD** *Companies: Amazon* A vector dimension or embedding-model mismatch between how the index was built and how the current query is being embedded — e.g. the index was built with one embedding model and queries are now being embedded with a different one.

IQ86. `model.generate()` runs but produces an empty string. What would you check first? **DIFFICULTY: MEDIUM** *Companies: Hugging Face* Whether an end-of-sequence/stop token is being generated immediately (e.g. due to a malformed prompt or an incorrectly applied chat template), and whether `max_new_tokens` is set to zero or another unintended value.

IQ87. A RAG system's retrieval quality dropped after re-ingesting the corpus with a new chunking strategy, with no other changes. What's your investigation order? **DIFFICULTY: HARD** *Companies: Anthropic, Startup* Compare retrieval metrics (Recall@k, MRR) on the same evaluation set before and after the change to confirm the regression is real; then inspect specific eval queries where the old chunking retrieved correctly but the new one doesn't, to identify whether chunk size, overlap, or boundary placement is the specific cause.

IQ88. Users report the RAG API is "slow" but average latency looks fine in your dashboard. What do you check next? **DIFFICULTY: MEDIUM** *Companies: Google* p95/p99 latency rather than the mean, and per-stage latency breakdown (retrieval vs. reranking vs. generation) to identify whether a specific subset of requests or a specific pipeline stage is responsible for the tail latency users are experiencing.

51.26 System Design / Scenario-Based

IQ89. Design a RAG system for 500 PDFs and 100 user questions, where answers must include page-level citations. What are the key design decisions? **DIFFICULTY: HARD** *Companies: Amazon, Anthropic* PDF extraction with per-page text and page-number metadata preserved through chunking; a chunk size that keeps citations meaningful at the page level; hybrid retrieval if the PDFs contain technical identifiers; a citation-validated generation prompt (Chapter 8.29); and an evaluation set built from realistic questions with known expected page-level sources.

IQ90. A corpus has documents at three different access levels (public, internal, confidential). How do you design retrieval to respect this? **DIFFICULTY: HARD** *Companies: Amazon, Google* Tag every chunk with an access-level in metadata at ingestion, and enforce filtering by the requesting user's access level at or before the similarity search itself — never only on the final generated answer (Chapter 8.38).

IQ91. Your RAG system needs to support both English and Spanish documents and queries. What changes relative to an English-only design? **DIFFICULTY: MEDIUM** *Companies: Startup* Evaluate and likely switch to a multilingual embedding model rather than an English-only one, and build a bilingual evaluation set — a model that performs well on English retrieval benchmarks can underperform significantly on other languages.

IQ92. How would you detect that your production RAG system has started hallucinating more frequently after a document corpus update, without waiting for user complaints? **DIFFICULTY: HARD** *Companies: Anthropic* Maintain a standing evaluation set with groundedness scoring (human or LLM-as-judge) and re-run it on a schedule or after every corpus/pipeline change, rather than relying solely on reactive user-reported issues.

IQ93. When would you choose long context over building a RAG pipeline? **DIFFICULTY: MEDIUM** *Companies: Google, OpenAI* When the document set is small, static, and consistently mostly-relevant to queries, and query volume is low enough that the per-request cost of processing the full context repeatedly isn't a concern — otherwise retrieval's targeted, cheaper per-request cost and better relevance-precision at scale generally wins.

IQ94. Why might you use RAG and fine-tuning together rather than choosing one? **DIFFICULTY: HARD** *Companies: Anthropic, Meta* Fine-tuning can adapt the model's behavior, style, or output format (e.g. always respond in a specific structured way), while RAG supplies current, private, or frequently-changing factual knowledge — the two solve different problems and are commonly combined in practice.

51.27 Coding

IQ95. Write a function that computes cosine similarity between two NumPy vectors without a library helper. **DIFFICULTY: EASY** *Companies: general*

```
import numpy as np
def cosine_similarity(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

IQ96. Write a retry decorator with exponential backoff for a function that calls an LLM API. **DIFFICULTY: MEDIUM** *Companies: Amazon*

```
import time, random, functools

def retry_with_backoff(max_retries=5):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return fn(*args, **kwargs)
                except (TimeoutError, ConnectionError):
                    if attempt == max_retries - 1:
                        raise
                    time.sleep((2 ** attempt) + random.uniform(0, 1))
        return wrapper
    return decorator
```

IQ97. Write a function that validates citation numbers in a generated answer against the actual retrieved chunk count. **DIFFICULTY: MEDIUM** *Companies: Anthropic*

```
import re
def validate_citations(answer_text, retrieved_chunks):
    cited = {int(n) for n in re.findall(r"\[(\d+)\]", answer_text)}
    valid = set(range(1, len(retrieved_chunks) + 1))
    return cited.issubset(valid)
```

IQ98. Write a function that builds a context string from ranked chunks without exceeding a token budget. **DIFFICULTY: HARD** *Companies: Google*

```
def build_budgeted_context(chunks, max_tokens, count_tokens_fn):
    selected, used = [], 0
    for c in chunks:  # assumed sorted by relevance descending
        t = count_tokens_fn(c["text"])
        if used + t > max_tokens:
            break
        selected.append(c); used += t
    return "\n\n".join(c["text"] for c in selected)
```

51.28 More Conceptual / Rapid-Fire

IQ99. What is the KV cache, at a practical (not derivational) level? **DIFFICULTY: MEDIUM** *Companies: Google, NVIDIA* A cache of previously-computed attention keys and values for earlier tokens in the sequence, reused rather than recomputed at each new generation step — this is what makes autoregressive generation of long sequences tractable instead of quadratically expensive per token.

IQ100. Why does quantizing a model to a lower precision typically preserve most of its quality? **DIFFICULTY:**

HARD *Companies: Meta* Trained weights typically have more numerical precision than is functionally necessary for most forward-pass computations; lower-precision formats (e.g. 8-bit, 4-bit) approximate the original values closely enough that output quality degrades only modestly for many tasks, though the exact quality cost varies by model and quantization method (full treatment in Part 9).

IQ101. What's the difference between a document's embedding and a chunk's embedding in a RAG system? **DIFFICULTY: EASY** *Companies: general* A document embedding represents an entire document as one vector (useful for document-level operations like deduplication or coarse routing); a chunk embedding represents one retrievable unit within that document — most RAG retrieval operates on chunk embeddings, not whole-document embeddings.

IQ102. Why is embedding caching for unchanged documents a "free" optimization? **DIFFICULTY: EASY** *Companies: Amazon* Re-embedding content that hasn't changed produces an identical result at real compute cost — caching by content hash avoids this redundant work entirely with no quality tradeoff.

IQ103. What does "grounded prompt" mean specifically, as a prompt pattern? **DIFFICULTY: EASY** *Companies: general* A prompt that explicitly instructs the model to answer only using supplied context, and to state explicitly when the context is insufficient, rather than falling back on its own parametric knowledge.

IQ104. Why is "answerability detection" a useful pre-generation step? **DIFFICULTY: HARD** *Companies: Anthropic* It estimates, before running the full (more expensive) generation call, whether the retrieved context actually contains an answer — allowing low-confidence cases to be routed differently (e.g. to abstention or human review) rather than always attempting generation regardless of retrieval quality.

IQ105. What's a practical threshold-based approach to abstention? **DIFFICULTY: MEDIUM** *Companies: Startup* Discard or trigger an "insufficient information" response when the top retrieved result's similarity score falls below a threshold tuned against your own evaluation set, rather than always proceeding to generation regardless of retrieval confidence.

IQ106. Why does duplicate document detection matter for retrieval quality, not just storage efficiency? **DIFFICULTY: MEDIUM** *Companies: Google* Duplicate content occupying multiple top-k slots wastes retrieval capacity that could have gone to genuinely different relevant information, and can skew an evaluator's or user's perception of "multiple sources agreeing" when it's really one source counted twice.

IQ107. What's the tradeoff between a larger k in retrieval and answer quality? **DIFFICULTY: MEDIUM** *Companies: Amazon* Larger k increases the chance of covering the genuinely relevant chunk (recall) but also increases context size, cost, latency, and the risk of diluting the model's attention with irrelevant content.

IQ108. Why is a corpus a moving target, and what does that imply for a RAG system's design? **DIFFICULTY: MEDIUM** *Companies: Startup* Documents get added, updated, and superseded over time; a RAG system needs an explicit re-ingestion/update strategy (and staleness-aware metadata) rather than being designed as if the corpus were a fixed, one-time snapshot.

IQ109. What is the difference between extraction and retrieval, precisely? **DIFFICULTY: EASY** *Companies: general* Extraction pulls raw text out of a source file format, happening once per document at ingestion time. Retrieval finds relevant pieces of already-extracted, already-indexed text, happening once per query.

IQ110. Why should you assert non-trivial extracted text length during ingestion? **DIFFICULTY: MEDIUM** *Companies: Amazon* To catch silent extraction failures early (e.g. a scanned PDF returning an empty string per page) rather than discovering, only at query time, that an entire document was never actually indexed with usable content.

IQ111. What's a concrete example of provenance metadata mattering for correctness, not just traceability? **DIFFICULTY: MEDIUM** *Companies: Anthropic* An ingestion timestamp or "superseded" flag lets a system prefer the current version of a policy document over an outdated one when both exist in the corpus, rather than presenting them as equally authoritative.

IQ112. Why is "fits within the context window" not the same as "the model will use it well"? **DIFFICULTY: HARD**

Companies: Google, Anthropic Models do not attend uniformly well across very long contexts in practice — retrieval of a specific relevant fact can degrade as irrelevant surrounding content grows, even when the total token count is technically within the supported window.

IQ113. What's the risk of a RAG system that always attempts an answer, with no abstention path? **DIFFICULTY: MEDIUM** *Companies: Startup* It will confidently answer even when the retrieved context doesn't actually support an answer, producing hallucinations that look identical in tone/confidence to genuinely grounded answers, making them harder for users to distinguish.

IQ114. Why measure embedding throughput separately from indexing throughput? **DIFFICULTY: MEDIUM** *Companies: Amazon* They have different bottlenecks — embedding is typically compute/GPU-bound (or API-latency-bound for hosted embedding models), while indexing (adding vectors to a FAISS/vector-DB structure) is typically memory/IO-bound; conflating them hides which stage actually limits ingestion speed.

IQ115. What is the risk of hard-coding a chat template string instead of using the model's own tokenizer config? **DIFFICULTY: MEDIUM** *Companies: Hugging Face* Special-token formats differ across model families and even across versions within a family; a hard-coded template can silently become wrong when the underlying model changes, producing degraded output with no obvious error.

IQ116. Why can't fine-tuning alone reliably solve a "the model doesn't know about our private, frequently-updated documents" problem? **DIFFICULTY: HARD** *Companies: Anthropic, Meta* Fine-tuning bakes patterns into fixed weights at a point in time and requires retraining (slow, expensive) to reflect updates — it doesn't give the model an efficient lookup mechanism for a large, changing document set the way retrieval does.

IQ117. What's the danger of evaluating a RAG system only on questions the corpus can definitely answer? **DIFFICULTY: HARD** *Companies: Startup* It never tests the abstention/failure path, so you learn nothing about whether the system correctly recognizes and communicates when it *can't* answer — which is often more important for user trust than performance on easy, well-covered questions.

IQ118. Why is a health check endpoint (`/health`) a minimum requirement for a deployed RAG API, not a nice-to-have? **DIFFICULTY: EASY** *Companies: general* Orchestrators and load balancers need a fast, reliable signal of service health to route traffic correctly and to trigger restarts of unhealthy instances automatically — without it, a broken instance can keep receiving traffic indefinitely.

IQ119. What's a reasonable first response when an interviewer asks "how would you reduce hallucination in our RAG system?" with no other context given? **DIFFICULTY: MEDIUM** *Companies: general* Ask what's currently measured (is there an evaluation set? what's the current groundedness rate?) before proposing fixes — grounding, citation validation, confidence thresholds, and abstention are all reasonable levers, but which to prioritize depends on where the current system is actually failing.

IQ120. Summarize, in one sentence each, when to reach for prompting, RAG, fine-tuning, and agents. **DIFFICULTY: MEDIUM** *Companies: general* Prompting for tasks solvable with instructions and context that fit in a single call; RAG for knowledge-grounded answers over private or frequently-changing document sets; fine-tuning for adapting behavior, style, or output format; agents for autonomous multi-step tasks requiring tool use and planning. # Chapter 8.51 — Master LLM/RAG Cheat Sheets

★ Cheat Sheet 1: LLM Inference

- Pipeline: text → tokenizer → token IDs → embeddings → transformer blocks → logits → decoding → text.
- Inference = forward pass only, no weight updates (contrast with training).
- Base model completes text; instruction-tuned model follows tasks; chat model handles multi-turn roles.
- Local inference: `transformers pipeline` for prototyping, `AutoModelForCausalLM` for production control.
- Key params: `torch_dtype` (precision/memory), `device_map` (device placement).

★ Cheat Sheet 2: Hugging Face Model Loading

- `AutoTokenizer.from_pretrained(model_id)` + `AutoModelForCausalLM.from_pretrained(model_id)`.
- Pin `revision` for reproducibility; repos change in place.
- Gated models require `token=` (a Hugging Face access token) after accepting the license on the Hub.
- `device_map="auto"` lets `accelerate` handle multi-GPU/CPU-offload placement automatically.
- `local_files_only=True` forces cache-only loading for offline/airgapped environments.

★ Cheat Sheet 3: Generation Parameters

- Greedy = always highest-probability token; deterministic, prone to repetition.
- Temperature scales logits before softmax: lower = sharper/more deterministic, higher = flatter/more random.
- Top-k: fixed-size candidate pool. Top-p: adaptive candidate pool by cumulative probability.
- Repetition penalty down-weights already-generated tokens.
- `max_new_tokens` caps generated length; distinct from total context window.
- `temperature=0` $\approx$ greedy decoding.

★ Cheat Sheet 4: Chat Templates

- Always call `tokenizer.apply_chat_template(messages, add_generation_prompt=True)`.
- Never hand-concatenate role strings — special tokens differ per model family.
- Roles: `system` (behavior/constraints), `user`, `assistant` (prior turns for multi-turn memory).
- `tokenize=False` to inspect the formatted prompt string; `tokenize=True` to get model-ready input IDs directly.

★ Cheat Sheet 5: Prompt Engineering

- Levers: clear instructions, context, constraints, output format, few-shot examples, role, decomposition, delimiters, negative constraints.
- Wrap user/retrieved content in explicit delimiters (triple quotes, XML tags) — separates instructions from data and mitigates injection.
- Zero-shot for simple tasks; few-shot when output style/format needs demonstrating.
- Grounded prompts (RAG): "answer using ONLY the context; say so if insufficient."

★ Cheat Sheet 6: Structured Outputs

- Pipeline: input $\rightarrow$ LLM $\rightarrow$ structured output $\rightarrow$ validation $\rightarrow$ application logic.
- Prompted JSON is not guaranteed valid — always validate, never trust raw parsing.
- Constrained decoding (serving-layer feature) makes invalid structure impossible, not just discouraged.
- Use Pydantic `model_validate_json()` and handle the failure path explicitly (retry/log/fallback).

★ Cheat Sheet 7: Embeddings

- Embedding model $\neq$ generative LLM — different training objective (contrastive vs. next-token).
- Output: one fixed-length dense vector per text.
- Normalize (unit length) at index-build time so inner product $\equiv$ cosine similarity.
- Validate embedding model choice on your own corpus/queries, never a generic leaderboard alone.

★ Cheat Sheet 8: Cosine Similarity

- Measures orientation (angle) between vectors, independent of magnitude — robust to text-length differences.
- `cos_sim(a, b) = dot(a, b) / (norm(a) * norm(b))`.
- On normalized vectors: cosine similarity ranking = inner product ranking (cheaper to compute).
- Implementations: NumPy (from scratch), `sklearn.metrics.pairwise.cosine_similarity`, `sentence_transformers.util.cos_sim`.

★ Cheat Sheet 9: FAISS

- `IndexFlatL2` = exact Euclidean distance search. `IndexFlatIP` = exact inner-product search.
- Normalize vectors before adding to `IndexFlatIP` for cosine-similarity-equivalent ranking.
- `index.add(vectors)` to build; `index.search(query, k)` returns `(scores, indices)`.
- Persist with `faiss.write_index(index, path)` / reload with `faiss.read_index(path)`.
- Flat indexes are exact but linear-cost — fine at small/medium scale, not for billions of vectors without an approximate index type.

★ Cheat Sheet 10: Vector Databases

- Core capabilities: storage, indexing, similarity search, metadata filtering, persistence.
- FAISS = fast library, no built-in persistence/filtering/server.
- Chroma = easiest to start with, small–medium scale. Qdrant/Weaviate = rich filtering, managed cloud options. Milvus = built for very large scale.
- Push filtering down into the search itself when possible — filtering only after search wastes top-k slots and, for auth, risks exposure.

★ Cheat Sheet 11: Chunking

- Arguably the single highest-leverage RAG decision.
- Recursive splitting (section → paragraph → sentence → word fallback) is the standard general-purpose default.
- Starting point: 300–800 tokens per chunk, 10–20% overlap — tune against your own eval set.
- Overlap prevents facts near chunk boundaries from being orphaned from context.
- Semantic chunking is more adaptive but more expensive — validate benefit before adopting.

★ Cheat Sheet 12: Metadata

- Attach per chunk: `source`, `page/section`, `document_id`, `chunk_id`, `timestamp`, `access_level`.
- Uses: filtering, citations, debugging, access control, provenance/staleness tracking.
- `document_id` should be stable and independent of filename.

★ Cheat Sheet 13: Retrieval

- Dense (embeddings) = semantic/paraphrase strength. Sparse (BM25/keyword) = exact-match strength.
- Hybrid = combine both, commonly via reciprocal rank fusion (score = $\sum 1/(\text{rank} + 1)$ across ranked lists).
- Larger `k` → higher recall, more context tokens, more noise — tune, don't maximize blindly.
- Filter by metadata before/during search when the search engine supports it, not only after.

★ Cheat Sheet 14: Reranking

- Pipeline: query → retriever → top 20 → reranker → top 5 → LLM.
- Bi-encoder (retriever): embeds query/doc separately, indexable, fast, scales to full corpus.
- Cross-encoder (reranker): joint query+doc input, one relevance score, not indexable, only for small shortlists.
- Retrieve wide, rerank narrow — bi-encoders can under-rank truly relevant documents that cross-encoders correctly promote.

★ Cheat Sheet 15: RAG

- Pipeline: load → clean → chunk → embed → index (ingestion) → retrieve → rerank → context → LLM → answer (query time).
- Build raw-Python RAG before adopting a framework — makes the underlying mechanics debuggable.
- Citation-aware RAG: number sources in-prompt, require citation by number, validate cited numbers in code.
- Advanced patterns (HyDE, multi-query, contextual compression, parent-child, hierarchical) — validate need before adopting; chunking/hybrid search fix most issues first.

★ Cheat Sheet 16: RAG Evaluation

- "Fluent answer" $\neq$ evaluation metric — evaluate retrieval and generation separately.
- Retrieval: Recall@k, Precision@k, MRR, hit rate.
- Generation: groundedness, answer relevance, completeness, citation correctness.
- Build eval sets from realistic user questions, not document paraphrases.
- Always compare at least two configurations against the same eval set before shipping a pipeline change.

★ Cheat Sheet 17: Hallucination Debugging

- Hallucination cannot be fully eliminated — only reduced and made visible.
- Levers: retrieval grounding, explicit context instructions, citations, confidence thresholds, abstention, answerability detection, retrieval validation.
- Diagnostic order: was the right chunk retrieved? was it within the token budget sent to the LLM? only then treat it as a generation-layer failure.
- Temperature/decoding strategy does not fix hallucination — grounding does.

★ Cheat Sheet 18: Context Management

- Reserve output tokens FIRST, then fill remaining budget with highest-relevance retrieved content.
- Count tokens with the target model's actual tokenizer — word/character estimates are unreliable.
- Truncate lowest-priority content first: lowest-relevance chunks, then oldest conversation turns.
- Summarization compresses and retains information; truncation simply drops it.

★ Cheat Sheet 19: Cost Optimization

- Highest-leverage lever: caching (eliminates redundant calls entirely, not just shrinks them).
- Other levers: batching, smaller/routed models, shorter prompts, fewer retrieved chunks, embedding caching, async concurrent calls.
- Streaming improves perceived latency only — does not reduce total cost or total generation time.
- Always measure p50 AND p95 latency, not just mean.

★ Cheat Sheet 20: LLM Security

- Indirect prompt injection = malicious instructions arriving via retrieved content, not typed by the user — the RAG-specific threat.
- Defenses: explicit delimiters around retrieved content, least-privilege on any downstream actions, output validation, provenance tracking, monitoring.
- Enforce authorization AT retrieval (before content reaches the LLM), never only on the final output.
- Content doesn't need malicious intent to pose injection risk — ordinary user-generated content can incidentally read as instructions.

★ Cheat Sheet 21: FastAPI RAG Deployment

- Load index/clients once at startup, not per request.
- Pydantic models for request/response validation on every endpoint.
- Explicit `/health` endpoint; structured logging; catch and log exceptions, return generic errors to clients.
- Config and secrets via environment variables, never hard-coded.
- Add a streaming endpoint variant for interactive clients when latency-sensitive.

★ Cheat Sheet 22: Docker RAG Deployment

- Pin dependency versions in `requirements.txt` for reproducibility.
- Pass secrets at `docker run` time (`-e`), never bake into the image.
- Bind the server to `0.0.0.0`, not `127.0.0.1`, inside the container.
- Add a `HEALTHCHECK` so orchestrators can detect and restart unhealthy containers.
- Mount index/data as volumes so they persist independently of the image and container lifecycle.

Chapter 52

Final Project Roadmap

```
LLM Basics
 ↓
Model Loading
 ↓
Inference
 ↓
Prompting
 ↓
Structured Output
 ↓
Embeddings
 ↓
Cosine Similarity
 ↓
FAISS
 ↓
Vector Databases
 ↓
Document Ingestion
 ↓
Chunking
 ↓
Retrieval
 ↓
Reranking
 ↓
RAG
 ↓
RAG Evaluation
 ↓
RAG Security
 ↓
FastAPI
 ↓
```

```
Docker
 ↓
Portfolio RAG System
```

By the end of this Part, you should be able to build — without relying on a framework to perform the underlying engineering for you — the complete chain:

documents → embeddings → vector search → retrieval → reranking → grounded LLM response → citations → evaluation → API → Docker.

That raw-Python fluency is exactly what makes Part 11's frameworks (LangChain, LangGraph, MCP) legible rather than magical: every abstraction they provide over retrieval, prompt assembly, and tool orchestration maps directly onto a pattern you've now built by hand.

Chapter 53

Transition to Part 9

Part 8 deliberately did not teach LoRA, QLoRA, PEFT, full fine-tuning, supervised fine-tuning in depth, RLHF, or DPO. Those are the subject of:

Chapter 54

Part 9 — LLM Fine-Tuning & PEFT

Part 9 will cover:

- Supervised fine-tuning and instruction tuning
- Training datasets and chat template construction for training
- The Hugging Face `Trainer` and custom training loops
- LoRA, PEFT, and QLoRA
- Quantization and `bitsandbytes`
- Gradient checkpointing and memory optimization
- Fine-tuning evaluation
- Catastrophic forgetting
- Model merging and adapter management
- Hugging Face Hub workflows
- Complete fine-tuning projects

Following Part 9:

Chapter 55

Part 10 — Open-Source Model Fine-Tuning & Contribution Engineering

Part 10 covers actual open-source contribution workflows — taking the fine-tuning skills from Part 9 into real upstream collaboration.

And after that, the agentic layer this Part deliberately deferred:

Chapter 56

Part 11 — Agentic AI, LangChain, LangGraph & MCP

Where the raw retrieval, prompting, and API patterns built in this Part become the substrate for autonomous, multi-step, tool-using systems.